myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System

SAABIRESH LETCHUMANAN

BACHELOR OF COMPUTER SCIENCE WITH HONOURS

2026

myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System

SAABIRESH LETCHUMANAN

A thesis submitted
in fulfillment of the requirements for the degree of
BACHELOR OF COMPUTER SCIENCE WITH HONOURS

School Of Computing and Digital Technology

UNIVERSITY MALAYSIA OF COMPUTER SCIENCE AND ENGINEERING

2026

DECLARATION

I declare that this thesis entitled "**myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System**" is the result of my own research, except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

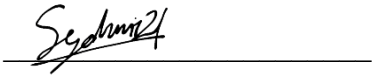
Signature: 

Name: SAABIRESH LETCHUMANAN

Date: 11 August 2026

APPROVAL

I hereby declare that I have read this thesis and in my opinion this thesis is sufficient in terms of scope and quality for the award of Degree BACHELOR OF COMPUTER SCIENCE WITH HONOURS

Signature: 

Supervisor: MUHAMMAD SYAHMIE SHABARUDIN

Date:

DEDICATION

To my beloved family, including my wife and parents, I would like to thank them for their continuous love, encouragement, and support throughout my academic journey. Their patience and understanding have been my greatest source of strength during challenging times.

I would also like to dedicate this work to my supervisor and lecturers, whose guidance, advice, and constructive feedback have played an important role in shaping this project. Finally, this thesis is dedicated to all friends on and off campus who offered help, motivation, and support throughout the completion of this study.

ABSTRACT

Parking challenges such as limited space availability, double parking, and inefficient parking management systems are common problems faced by drivers in busy urban areas in Malaysia. Most existing parking applications focus mainly on digital payments and often lack real-time parking information, parking reservation features, and effective communication tools to manage double-parking situations. To address these issues, this project proposes myMove, a smart mobile-based parking platform designed to improve the overall parking experience through a centralized and user-friendly system. The application focuses on features such as viewing parking information, reserving parking spaces, digital payments, booking management, and QR code–based communication between drivers without exposing personal contact details. This study was carried out during the FYP1 phase and involved a literature review and a pilot survey to identify user needs and limitations of existing systems. The findings indicate strong user interest in an integrated parking management solution and provide a solid foundation for the development, implementation, and evaluation of the full system in FYP2.

myMove: Sistem Tempahan Parkir Pintar Berasaskan IoT dan Komunikasi Kenderaan Berprivasi

ABSTRAK

Masalah parkir seperti kekurangan tempat letak kereta, parkir berganda, dan sistem pengurusan parkir yang tidak cekap merupakan isu yang sering dihadapi oleh pemandu di kawasan bandar yang sibuk di Malaysia. Kebanyakan aplikasi parkir sedia ada lebih tertumpu kepada pembayaran digital dan sering kekurangan maklumat parkir masa nyata, ciri tempahan parkir, serta alat komunikasi yang berkesan untuk mengurus situasi parkir berganda.Bagi menangani masalah ini, projek ini mencadangkan myMove, iaitu sebuah platform parkir pintar berasaskan aplikasi mudah alih yang direka untuk meningkatkan pengalaman parkir secara keseluruhan melalui sistem yang berpusat dan mesra pengguna. Aplikasi ini memfokuskan kepada ciri-ciri seperti paparan maklumat parkir, tempahan ruang parkir, pembayaran digital, pengurusan tempahan, serta komunikasi berasaskan kod QR antara pemandu tanpa mendedahkan maklumat peribadi.Kajian ini dijalankan pada fasa FYP1 dan melibatkan kajian literatur serta tinjauan perintis bagi mengenal pasti keperluan pengguna dan kekangan sistem sedia ada. Dapatan kajian menunjukkan minat yang tinggi terhadap penyelesaian pengurusan parkir yang bersepadu dan menyediakan asas yang kukuh untuk pembangunan, pelaksanaan, dan penilaian sistem penuh dalam fasa FYP 2.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for the continuous guidance, valuable advice, and constructive feedback provided throughout the completion of this Final Year Project. Their support and encouragement were essential in helping me stay focused and motivated during the research and documentation process. I would also like to thank the lecturers and academic staff for sharing their knowledge and providing helpful insights that contributed to the development of this project. Special thanks are extended to all participants who took part in the pilot study survey, as their time, responses, and honest feedback were invaluable in identifying real-world parking challenges and shaping the requirements of the proposed system. Finally, I would like to express my deepest appreciation to my wife, my family, and my friends for their constant support, patience, and encouragement throughout this journey. Their understanding and motivation played a significant role in helping me complete this project successfully.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Full Term
2FA	Two-Factor Authentication
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
APK	Android Application Package
BaaS	Backend as a Service
BLE	Bluetooth Low Energy
CRUD	Create, Read, Update, Delete
DBKL	Dewan Bandaraya Kuala Lumpur
ERD	Entity Relationship Diagram
ESP32	Espressif Systems 32-bit Microcontroller
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
HC-SR04	Ultrasonic Distance Sensor Module
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
LED	Light Emitting Diode

LPR	License Plate Recognition
MYR	Malaysian Ringgit
NoSQL	Non-Relational SQL / Not Only SQL Database
OTP	One-Time Password
P2P	Peer-to-Peer
QR	Quick Response Code
REST	Representational State Transfer
RFID	Radio Frequency Identification
RTDB	Firebase Realtime Database
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SOS	Emergency Panic Distress Signal
SSL	Secure Sockets Layer
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UAT	User Acceptance Testing
UI	User Interface
UID	Unique Identifier
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1
INTRODUCTION

1.1 Background

In recent years, searching for parking has become increasingly stressful because people who own vehicles are rising day by day (Chen et al., 2025). High density areas such as condominiums, shopping malls and office or university campuses face these issues endlessly. Facilities like visitor parking, resident parking and loading bays must operate efficiently to ensure smooth traffic flow and reduce congestion (Alshareef., 2024). However, issues such as double parking, difficulty finding parking spots, and long waiting times often cause frustration for drivers. Looking at today's technologies, many individuals must have their own mobile phone that allows them to access the internet or download numerous applications (Khalid et al., 2024).

Despite improvements in modern technology, many parking lots continue to use manual checking, basic systems of tickets, and cards that cannot provide real-time data (Puspitasari et al., 2021). Such systems miss critical functionalities like online reservation, availability updates, and cashless payment. Therefore, it takes much time for people to search for parking in the same area. The most important feature of the current project is to develop an easy-to-use mobile app called myMove which will let people find out whether there is available parking, book parking, pay for it via online method, and solve double-parking issues with QR Code and in-app communication.

1.2 Problem Statement

However, contemporary methods of urban parking management face a number of major challenges within modern cities. The challenge for drivers is finding parking spots because the system used is inefficient and does not give up-to-date information about their availability (Kalašová et al., 2021). For example, a typical parking area in Malaysia which forces drivers to continuously circle the premises in search of an unoccupied parking spot, leading to inefficiency and congestion. When drivers keep searching for a parking spot even though none are available, it causes stress and wastes their time, which can reduce their overall productivity. Besides, many parking systems in Malaysia are outdated and cannot keep up with the growing number of vehicles. Because they do not provide real-time information, drivers spend more time searching for parking, which increases traffic congestion and reduces the overall quality of urban mobility.

Additionally, double-parking remains a persistent issue as well in Malaysia, especially in crowded areas, where blocked vehicle owners have no fast or secure way to contact the driver who parked behind them (Haslinda et al., 2016). Despite DBKL's implementation to clamp cars that park in illegal parking areas in 2016, recent studies show that the number of compounds issued has increased dramatically between January to May 2025, which DBKL also received 1,009 public complaints related to illegal parking (Anis, 2025). This shows that current enforcement methods alone are not enough to solve the ongoing illegal and double-parking issues.

1.3 Research Question

This study aims to answer the following research questions:

i. How can a mobile application improve the way drivers find and reserve parking spaces without relying on physical hardware?
ii. How does QR-based communication help drivers handle double-parking situations?
iii. Can real-time applications reduce search time for drivers in busy urban areas?
iv. Can a dedicated application like myMove effectively solve the problems in real world situations and existing platforms?

1.4 Research Objective

The aim of this project is to develop a parking system that provides real-time information, finds parking spaces easily, reserves a spot and communicates during double-parking situations to reduce stress, search time and congestion.

The objectives of the project are as follows:

i. To study existing parking systems, identify current issues faced by Malaysian drivers, analyze the limitations of current car parking application, define user requirement specification and features through literature review and survey
ii. To develop the myMove application (mobile and web admin) by implementing its core features, including real-time parking search, hardware-integrated spot monitoring, and booking.

iii. To evaluate the application usability and user satisfaction by conducting User Acceptance Testing (UAT) with real users and collecting feedback for further improvement.

1.5 Scope of Research

The scope of this research is as follows:

i. Development of a mobile application (Android) using Flutter and backend with Firebase for authentication.
ii. This application targets Malaysian drivers who had overwhelming and frustrating experiences during their daily search for parking or dealing with double-park situations.
iii. Users will be able to create an account, which prompts them to enter details such as name, age, email, password, car number plate

1.6 Thesis Outline

Based on the objectives previously presented and, on the approach proposed before, this thesis is made up of five (5) chapters, which contents are summarized as follows:

- Chapter 1 (Introduction)
  - This chapter presents an overview of the project, including the background of the study, problem statement, objectives, scopes, contributions and significance of the research.

- Chapter 2 (Literature Review)
  - This chapter provides a review of existing literature related to the project and mobile parking applications. This chapter examines the current applications such as Touch ‘n Go eWallet, JomParking and Flexi Parking.

- Chapter 3 (Methodology)
  - This chapter explains the research methodology applied in this study. It outlines the research approach, data collection methods and the software development process that will be applied for this project.

- Chapter 4 (Result and Discussion)
  - This chapter presents the results and findings obtained from the pilot study and prototype development of this project. It includes survey results, user feedback and analysis related to current parking issues and user expectations.

- Chapter 5 (Conclusion and Recommendation for Future Search)
  - This chapter concludes the study by summarizing the main findings and contributions of the research. It also discusses the limitations of the current study and provides recommendations for future work, particularly for the full system development and evaluation in FYP2.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

In recent years, technological advancements have significantly transformed various aspects of daily life (Aditya Kumar 2022). This chapter will conduct an investigation of current research, studies and technologies with mobile application in parking management, parking detection in real-time and driver's communication. The research will consider the current problems experienced by drivers in Malaysia such as double parking, outdated parking system / application and lack of real-time availability of parking. Current solutions, namely Touch 'n Go eWallet in-app parking, JomParking and Flexi Parking will be analyzed in order to reveal their strength. This research will determine the necessity of creating a new and updated parking management system with features such as availability of parking lots in real-time, parking lot reservation, digital payment, number plate detection, driver's communication within the app, notification of car blocking, navigation to parking lots and QR-based contact system for emergencies.

2.2 Review of Academic Literature

To better understand the current landscape of mobile based parking management and communication systems, three relevant studies and existing solutions were reviewed. These studies helped identify key design features and gaps that inform the development of myMove.

Table 2.1 Comparative Study Of Car Parking System

Source	Main Findings	Methodology	Strengths	Weaknesses	Relevance to Project
P. Sampath, et al. IoT- based Smart Parking System for Industry 4.0 with QR code access and real- time navigation (2025)	The study shows that using IoT sensors, QR codes, and a mobile app can reduce parking search time, congestion, and user frustration, while improving parking efficiency and security.	IoT sensors, QR codes, and a mobile app can make parking faster, safer, and less stressful.	Offers real-time parking info, QR-based access, reservations, navigation, and a working hardware-software prototype.	The system is more suitable for controlled environments like malls or campuses but difficult to scale without infrastructure support.	This study highlights myMove's key features: real-time parking info, reservations, QR-based interaction, and mobile app use.
Mahat, S. N. & Nadzir, N. M. "Accessing User Satisfication of Smart Parking Payment System in Selangor" (2025)	User satisfaction in parking apps mainly depends on how easy they are to learn and how clearly information is shown, making usability more important than having many complex features.	A survey conducted among local users with statistical analysis to identify the factors that influence their satisfaction.	Uses data from Malaysian users to provide clear insights and recommendations on user experience.	Focuses on payment features rather than driver-to-driver communication or QR code sticker interactions.	Supports the importance of having a simple user experience, clear information, and features adapted in MyMove.
Balqis Lim "Uni students develop smart system to curb double parking problems (ParkKing)" (2017)	A smart outdoor parking system aimed at reducing double-parking by detecting illegally parked vehicles and enabling more efficient parking supervision	The project developed a ParkKing prototype using sensors to detect and manage double-parking in outdoor lots.	The system helps reduce double-parking by identifying problem parking and using technology to support enforcement and reduce traffic issues.	It lacks technical details such as system accuracy, implementation scope, real-world testing, and driver communication, focusing more on the announcement than evaluation.	This study shows a real-world effort to address double parking and highlights the need for better parking systems. It supports myMove's use of technology to manage illegal parking and shows the potential of smart detection systems in parking solutions.

2.3 Analysis of Commercial Applications

This section explains about the functionality and limitations of existing platforms used by Malaysian drivers. Below are three applications, Touch ‘n Go eWallet’s parking, JomParking and Flexi Parking were chosen because of their popularity.

2.3.1 Touch ‘n Go eWallet’s parking

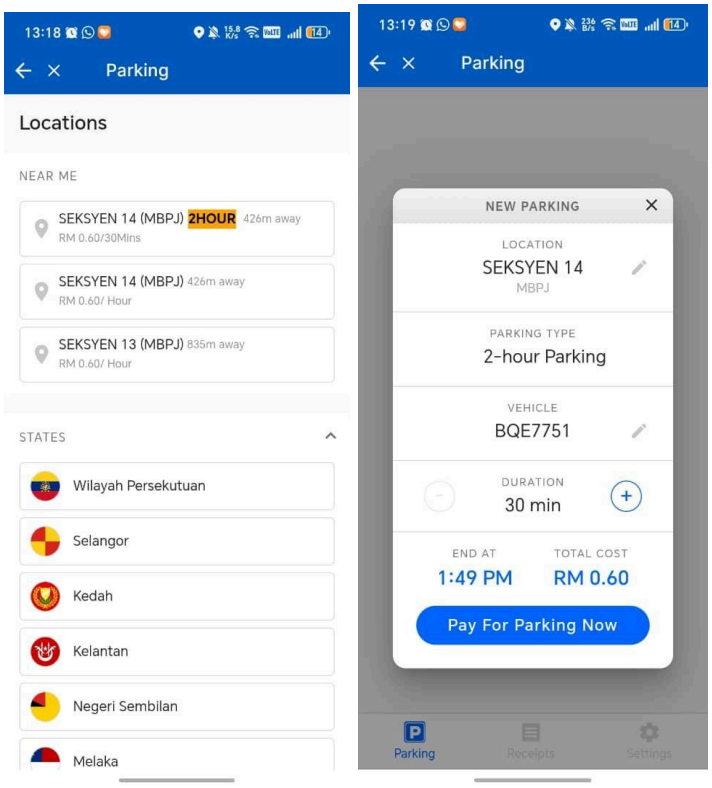


Figure 2.1: Example of Parking System in Touch ‘n Go eWallet.

Touch ‘n Go eWallet parking is a feature inside the Touch ‘n Go eWallet app that allows users to pay for parking digitally only at selected locations / street parking. The app is convenient because there is no need to use physical tokens, it automatically deducts parking fees from their eWallet balance and has LPR Parking feature. However, there are certain features that have limitations and lack features. It does not show real-time parking availability, so drivers still need to drive around to find an empty spot.

2.3.2 JomParking

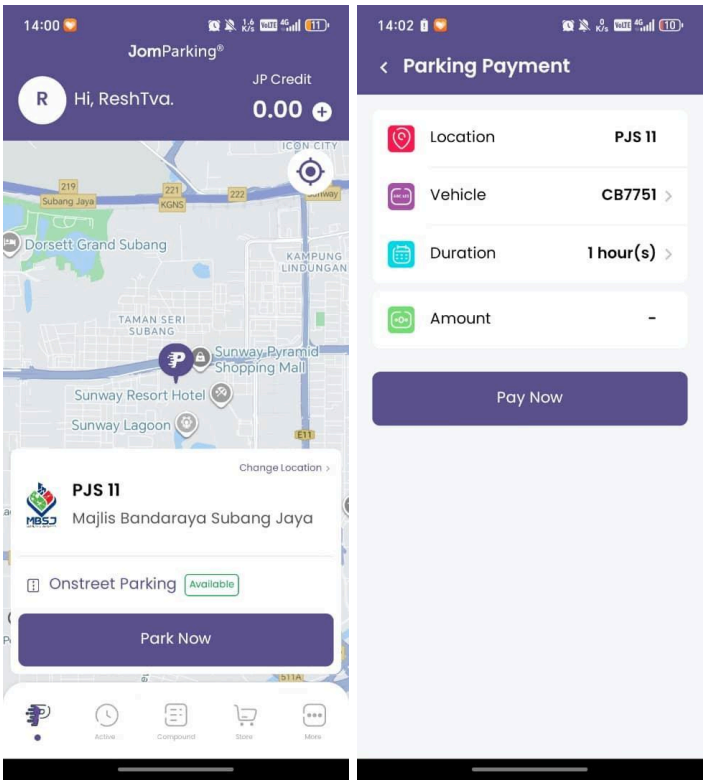


Figure 2.2: Example in JomParking

JomParking is a mobile parking application widely used in Malaysia as Touch ‘n Go eWallet to pay for street and city council parking. The app allows users to select their location by dragging the map, entering their car number plate and selecting the duration of the parking time. It also provides a city council compound, where users can pay for outstanding payments and includes parking passes, reload credit to the app and services like car battery delivery, booking a trip and car and motorbike insurance. However, their platform is basic and offers limited interaction. It doesn’t support every city council parking around Malaysia and doesn’t include every off-street parking like malls or buildings.

2.3.3 Flexi Parking

Figure 2.3: Example of Flexi Parking

Flexi Parking is a local platform in Malaysia that offers to pay for street parking under supported city councils. Users can select their parking zone, add durations, their car number plate and pay using in-app credit. However, it's not fully operational and optimized in terms of parking experience. The platform sometimes runs into an error, like connection issues, payment and top out problems, and receiving suspicious messages about traffic fines.

2.4 Suggested Approach

The proposed system is a mobile application called myMove, designed to improve the parking experience for Malaysian drivers by providing clearer information, easier communication and a more organized parking process. myMove focuses on giving drivers real-time parking availability with information, a reservation system, simple digital payment and a faster way to deal with double-parking.

The system allows users to view available parking spots in a building, select a space with details such as floor and parking number, and reserve it before entering. Once a spot is chosen, it becomes unavailable to others, reducing confusion and unnecessary searching around the same place. The app also sends reminders when the user's parking time is ending to help avoid extra charges. Additionally, a QR-based double-parking communication feature lets blocked drivers scan a displayed QR Code to contact the car owner through in-app messaging or calls without revealing personal phone numbers.

2.5 Benchmark with Existing Systems

The comparative analysis of existing systems and the proposed myMove application is detailed in the table below:

Table 2.2: Comparison Of Existing System

App Name	**Functionality**	**Strengths**	**Weaknesses**	**Proposed Improvements**
Touch ‘n Go eWallet	It has parking features, such as LPR parking, street parking and provides travel passes, named My50 Pass.	Convenient and fast payment method while using LPR parking and reduces physical cash or card.	The balance in the physical card and eWallet balance lacks integration and support limited location only.	Aligns with myMove goal and expands to more locations.
JomParking	Payment for street parking by providing city council’s, allow by user by dragging the map and offer compound payment.	Convenient and time-saving features like paying for parking remotely and extending their time.	Supports only limited locations, provides only certain compound payment and only has online banking.	Aligns with myMove goal to show parking details and parking space selection
Flexi Parking	Users can pay for on-street parking via GPS, receive expiry reminders, extend parking remotely, and manage up to 10 vehicles in one account.	Supports over 46 municipal councils, monthly parking passes and prevents users from paying during non-chargeable hours.	App hard to navigate, buggy, errors when registration, provides certain locations and inaccuracy GPS	Aligns with myMove goal to show parking details and parking space selection

2.6 Summary

This chapter examined existing parking applications used in Malaysia to identify their features, strengths and weaknesses. Applications such as Touch ‘n Go eWallet Parking, JomParking and Flexi Parking were reviewed and found to mainly focus on digital payment while it lacks comprehensive parking information, provides limited coverage, minimal user interaction and lack of support for double parking situations. These findings justify the need for the proposed myMove application as more integrated and user-friendly parking solutions.

CHAPTER 3
METHODOLOGY

3.1 Introduction

This chapter outlines the research methodology that addresses the Objective 1 of myMove project, which focuses on examining the limitations of current car parking applications, studying existing parking solutions and identifying user needs before developing the proposed system. This research approach is used to clearly understand the problems and define the system requirements. The Waterfall Model was chosen as the software development method because the project has clear requirements and follows a planned development process. The model follows a step-by-step approach, where each phase is completed before moving to the next phase. For Objective 1, the research includes a literature review and a user survey to study existing parking applications, understand user problems, and identify gaps in current parking systems. The later phases, including system design, development, testing, deployment, and maintenance, are also planned according to the Waterfall Model.

3.2 Proposed System Development Process

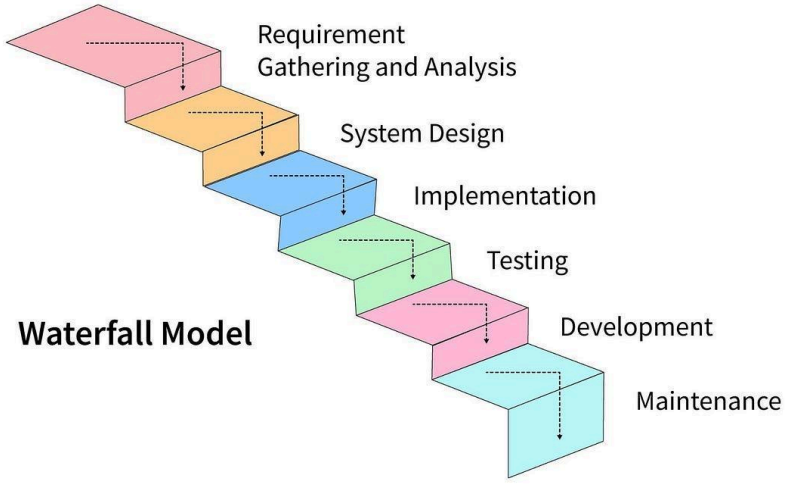


Figure 3.1 Waterfall Model that will be used for myMove project

Development of the project will take place based on the Waterfall Model where there is a sequential flow from the very first study of requirements up to the last maintenance of the system. Waterfall Model is selected for myMove as there are stable and clear requirements that fit the step-by-step approach of development with no questions. Sequential completion of each phase makes the development process much easier. Below is shown the application of each phase of the Waterfall Model during the system development:

- Requirements Phase:
  - This phase was completed in FYP1, where information about parking problems, user needs, and system requirements was collected and analyzed. Existing parking solutions were also studied to identify their limitations, such as the lack of real-time parking availability, limited information, and limited support for handling double-parking situations. Feedback from the pilot study was used to define the functional and non-functional requirements that guide the system design and development in the next phase.

- Design Phase:
  - The design phase focuses on turning the system requirements into a clear system plan. This includes creating use-case diagrams and UI prototypes to show how users will interact with the application. The UI designs created in FYP1 focus on simplicity and ease of use to show the main system flow, while more detailed and complete designs are developed in FYP2.

- Development Phase:
  - The development phase will take place in FYP2, where the myMove application will be developed based on the approved system design. The main features will be developed step by step, with a focus on keeping the code simple and easy to maintain. This phase will also connect the frontend and backend to ensure data is transferred correctly and all system features work as expected.

- Testing Phase:
  - Testing will be carried out throughout FYP2 to make sure the system works properly and meets the user requirements. Functional testing will be used to check that each feature works correctly, while usability testing will be used to see whether users can interact with the system easily and understand the interface. Any problems or errors found during testing will be fixed before the system is deployed.

- Deployment Phase:
  - After development and testing are completed, the application will be deployed in a controlled environment for demonstration and evaluation. This phase allows the system to be tested as a complete working prototype and provides feedback from users for further improvements.

- Maintenance Phase:
  - The maintenance phase focuses on improving the system after deployment. Feedback from testing and evaluation will be used to fix problems, improve performance, and add new features. This phase also considers making the system easier to expand and support future updates.

3.3 Gantt Chart

Activity	September	October	November	December
Problem Statement & Topic Approval	✔			
Literature Review	✔	✔		
Pilot Study & Survey Data Collection		✔	✔	
Data Analysis & Findings			✔	
System Requirements			✔	✔
Use Case Diagram			✔	✔
UI Prototype Design			✔	✔
FYP1 Thesis Submission				✔

Figure 3.2 Gantt Chart of Project Timeline

3.4 Limitations and Features

For developing myMove, it was necessary to determine the limitations of the existing car parking system and required features of the mobile application based on literature review and pilot study results.

3.4.1 Limitations of Current Systems

- **Lack of Mobile Accessibility:** Many shopping malls and structures use older systems that are not compatible with mobile applications, making it inconvenient for people to search for their parking or view how much time they spend.
- **Lack of Hardware Integration:** When parking lot availability depends upon manual or no hardware system at all.
- **Missing Parking Info:** In the existing application, there is no feature for building, floor, and parking space number information
- **Absence of Double-Parking Solutions:** The current system does not offer an option for handling double parking.
- **Lack of Integrated Communication Features:** Existing systems do not support in-app messaging and calling features for double-parking situations, forcing users to rely on external methods that may compromise privacy.

3.4.2 Proposed Features for myMove

Based on the pilot study with 44 people, that include university students and other individuals, the following features were identified as crucial for myMove to address these limitations:

- **Real-Time Parking Availability**: Allowing users to view available and occupied parking spaces in buildings and help drivers make faster parking decisions.
- **Parking Reservation**: Enabling users to reverse specific parking spaces in advance to reduce unnecessary searching.
- **Navigation Assistance**: Providing guided directions to the selected parking area to improve user and reduce travel time .
- **Parking Details:** Displays information such as parking fee rate, building name, floor level, parking space number and parking duration once the car is parked.
- **Emergency Mode:** Providing quick-access features that allow users to send urgent alerts in emergency situations.
- **Space for User Feedback:** Receive feedback from users / drivers about their experience in the specific place where they park so that we can improve.
- **QR Code-Based Solution for Double Parking:** Allow blocked drivers to scan a QR Code to contact car owners quickly without exposing confidential details.
- **In-App Messaging and VoIP Calling:** Enable secure communication between drivers through chat, and voice call within the application.

3.5 Data Collection Methods

To investigate the challenges faced by drivers and to gather user requirements for the parking system, the following methods were used:

Literature Review:

- **Source**: Existing mobile parking applications and publicly available information, including Touch ’n Go Parking, Flexi Parking, and JomParking, were reviewed to understand their features and how current parking systems work.
- **Focus:** The review focused on finding common limitations in existing systems, such as the lack of real-time parking availability, limited communication between users, and limited connection between parking information and user communication features.
- **Process:** Information was collected by reviewing the features of existing applications, official app descriptions, user reviews, and observations made between 2020 and 2025. The information was then compared to identify gaps and possible areas for improvement.

Pilot Study:

- **Participants:** Total of 37 participants including university students, and working adults aged between 18 to 45 above have taken part in the online survey to share their experiences and consent about car parking systems
- **Method:** Google Form was used to collect responses on difficulties finding parking spaces, delays caused by blocked vehicles and user expectations for a smart parking application.
- **Data Collection**: The results indicate that most respondents had trouble in finding available parking spaces (56.8%) and encountered double parking situations (89.2%). Most participants expressed strong interest in features such as showing vehicle parking details (73%), digital payment integration (62.2%) and QR code that helps blocked drivers (51.4%) as a main feature.

3.6 Develop Module of myMove and Integration

While the development phase will take place in FYP2, this section outlines the approach for bringing together myMove core components based on the requirements gathered in Objective 1. myMove will be developed as a cross-platform mobile application using Flutter with Dart for the frontend. Although Flutter supports both Androids and iOS, the first release of myMove will be limited to Android devices only due to current hardware and incompatibility with Apple devices. The interface will be designed using Figma, focusing on clean and minimal layout and user-friendly experience.

The integration plan covers:

- **Frontend Development:**

Flutter will be used to develop an intuitive and responsive user interface compatible with Android devices. The frontend will include important key features such as:

- User login / registration (with secure authentication)
- Displays an interactive map for exploring nearby parking locations.
- A search bar for destinations and a bottom navigation to profile, QR scanner for double parking situation and history
- Shows a building-level parking layout with color-coded indicators for available, occupied, and reserved spaces.
- Users can select a parking spot, and the system updates its status when they arrive and extend parking duration
- Shows parking details (floor, section, lot, duration), lets users upload a photo, and provides fee info with instant payment.

- **Backend Development:**

The backend will be powered by Firebase that includes various product categories like authentications and more. The core backend responsibilities include:

- Managing authentication (email, password, or Google sign-in) using Firebase Authentication
- Deliver instant push notification using Firebase Cloud Messaging (FCM)
- Manage real-time parking status updates using Firebase Realtime Database
- Support media uploads such as parking spot photos using Firebase Storage
- Location based parking search and navigation guide using Google Maps API
- Server-side processes such as payments and reservations using Cloud Functions.
- Generate and scan QR codes using Flutter QR libraries (qr_flutter)
- Communication and calls using Firebase Realtime DB and third-party API's
- Payments will be handled using Stripe, Touch 'n Go eWallet, or any supported Malaysian gateway integrated through Firebase Functions.

- **Database Structure:**
  - Storing user details, building, floor and parking lot data for map and location retrieval using Firebase Firestore
  - Manage communication records (chat messaging and call) using Agora

- **Hardware Integration:**
    - Parking sensors will be installed in individual parking spaces to detect whether a spot is occupied or vacant using ESP32 microcontrollers.
    - Each ESP32 device will transmit parking status data wirelessly to the backend system through Wi-Fi and sync with Firebase Firestore.

3.7 Summary

This chapter describes the methodology used for the myMove project, where the Waterfall Model was applied to guide system development in a clear and structured way. It describes how user needs and system limitations were identified through a literature review and pilot study carried out during FYP1. The chapter also presents the proposed features and overall system, including frontend, backend, database and hardware. In addition, a Gantt chart is included to show the planned project timeline and development schedule.

CHAPTER 4

DEVELOPMENT & IMPLEMENTATION

4.1 Introduction

The following Waterfall method is shown in Chapter 4, this phase is where we will transform system requirements to a prototype development. The goal in this phase is to connect mobile application development (frontend and backend), IoT hardware, and cloud services into one single platform that supports real time parking availability, parking reservation, digital payments and effective parking management. The system is categorized into three different layers which is IoT Hardware Layer, Cloud Backend Layer, and the Application Layer. The IoT Hardware layer that uses ESP32 and HC-SR04 sensors to detect vehicles on parking slots. The cloud uses Firebase including Firebase RTDB for real-time updates, Cloud Firestore for data storage and Firebase Authentication for user management. On the application layer, it provides a Flutter mobile application that allows users to check parking availability, reserve parking spaces, make digital payments, contact vehicle owners using QR codes and emergency SOS features. A website dashboard also used React.js with Vite to allow admins or parking management to monitor parking occupancy, facilities and view emergency alerts. This chapter describes the system functional and non-functional requirement, system architecture, hardware setup, database structure, software components and lastly summary of the developed achievements.

4.2 Functional Requirement

This section shows the functional requirement of myMove, developed from user needs gathered during the requirement analysis phase.

- **User Authentication and Session Management**: Secure user registration, login, profile management, and persistent user sessions in the Flutter mobile application using Firebase Authentication.
- **Vehicle & QR Code Management:** CRUD management of registered vehicles (license plate, model, color) and automatic generation of unique, high-resolution contact QR codes stored and retrieved via Firebase Storage.
- **Real-time Parking Spot Grid:** Dynamic, visual grid layout representing available and occupied spots. The grid updates immediately in response to live sensor states utilizing stream listeners connected to the Firebase Realtime Database (RTDB).
- **Parking Spot Reservation & Booking:** Spot selection, time duration setup, price breakdown calculation, and confirmation of booking status persisted in Cloud Firestore.
- **Payment Gateway Integration:** Secure processing of credit card transactions for reservations using Stripe API and SDK integrations.
- **Ultrasonic Occupancy Detection:** Proximity monitoring of physical parking spots using HC-SR04 ultrasonic sensors controlled by an ESP32 microcontroller to identify vehicle presence.
- **Local Status Feedback:** Hardware status indicators using dual LEDs connected to the ESP32 (Red for occupied slots, Green for vacant slots) to provide immediate feedback to drivers in the physical parking lot.
- **Live Database Synchronization:** Automatic transmission of physical slot changes to the cloud from the ESP32 utilizing HTTP requests/REST APIs connected to the Firebase RTDB.
- **SOS & Emergency Broadcast:** One-tap emergency/panic trigger in the mobile app, instantly transmitting user profile details and current GPS coordinates to the Admin Dashboard for operator action.
- **Web Guest Contact Flow:** Web-based portal (scanner-web) allowing non-app users (guests) to scan a vehicle's QR code, verify the vehicle's public details, and request a vehicle move.
- **Blocked-Vehicle Alerts & Move Requests:** Ability to upload proof photos and share GPS coordinates via the web guest browser to report blocking vehicles.
- **Real-Time Anonymous Chat:** Seamless, real-time messaging between a blocked driver (using the web guest portal) and the blocking car owner (using the Flutter in-app chat) to coordinate relocation without exposing personal contact details.
- **Web Admin CRUD Portal:** Administrative management allowing operators to create, read, update, and delete parking locations, zones, and individual spot-to-sensor mappings via the Web Admin Dashboard (admin-web).

- **Emergency Alert & Live Operations Hub:** Real-time console for admins to monitor triggered SOS emergency alerts, review active bookings, and broadcast system-wide announcements to all mobile app users.
- **User Feedback and Bug Logging:** Submission and management of user experiences, rating logs, and issue reports within the system database.

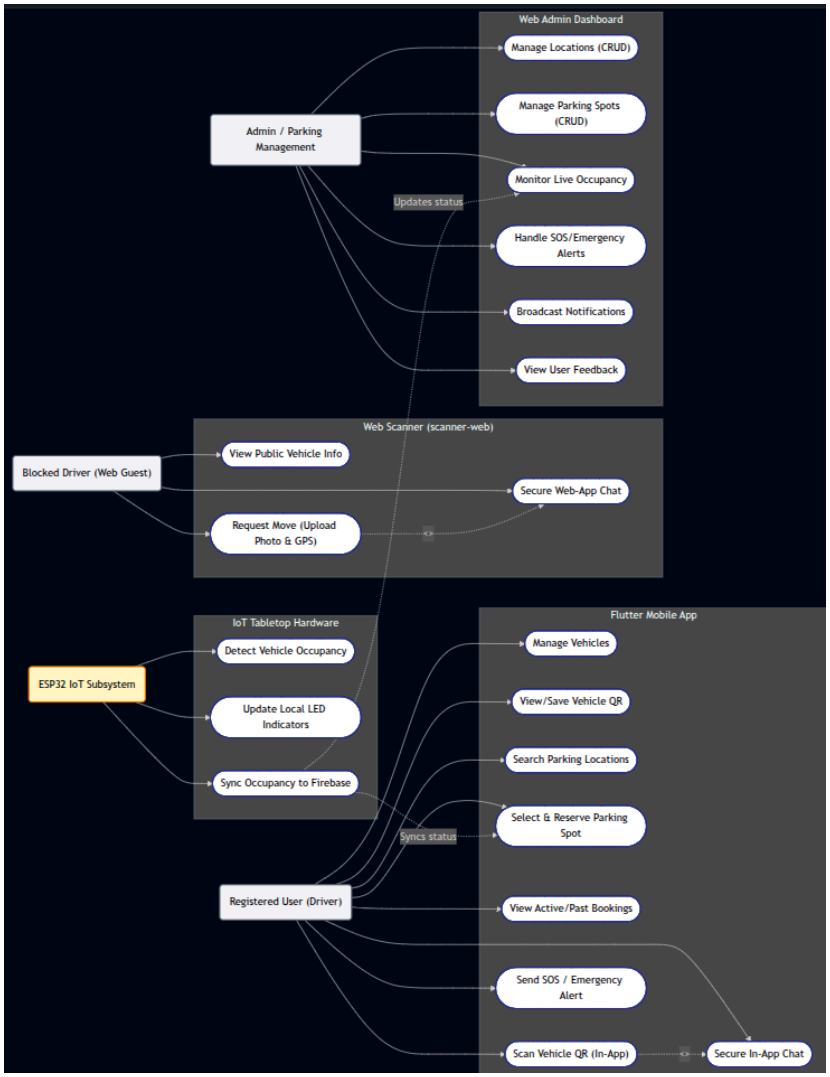


Figure 4.1: Use Case Diagram

4.3 Non-Functional Requirement

- **Security & Data Privacy:**

- Secure password protection and token-based session management are handled using Firebase Authentication. Access control is managed through custom Cloud Firestore Security Rules to ensure that users can only read and update their own profiles, vehicles, and bookings. Communication between users remains anonymous, as phone numbers and email addresses are never shared, and messages are securely connected through the vehicle's public QR identifier.

- **Performance & Real-Time Latency:**

- Real-time updates for parking spot availability from the ESP32 / HC-SR04 sensor to Firebase RTDB and driver's mobile application must be updated within 1.5 seconds. Chat messages sent between web guest scanner and mobile users must be delivered within 1.0 seconds under normal network conditions.

- **Web Guest Optimization:**

- The web guest scanner interface must load and ready to use on normal mobile browsers within 2.0 seconds without require to download apps, large files, and registration

- **Reliability & Hardware Fault Tolerance:**

- The ESP32 hardware includes an automatic Wi-Fi and Firebase reconnection to recover from connection problems in case any network problem occurs, without manually resetting any hardware. Sensor readings are stable by using multiple sensor readings (HC-SR04) to provide accurate availability detection and reduce false detection caused by obstructions.

- **Transactional Integrity (Concurrency):**

- Parking reservations must be processed using Firestore Transactions to ensure that each booking is handled safely and to prevent multiple users from booking the same parking slot at the same time.

- **Scalability & Database Efficiency:**

- The database structure organizes parking spots by location_id and zone_id in Cloud Firestore and Firebase RTDB, allowing the system to support thousands of parking spots without causing slow performance. Database indexes are optimized to maintain fast data retrieval.

- **Usability & Layout Responsiveness:**

- Web interfaces use responsive layouts that automatically adjust to mobile, tablet, and desktop screen sizes. The blocked driver process is designed as a simple three-step process (Scan QR → Upload Photo/GPS → Start Anonymous Chat) to make the process quick and easy for users.

- **Compatibility & Platform Support:**

- The Flutter mobile application supports Android (API Level 21 / Android 5.0 and above) while web interfaces are also fully compatible with Google Chrome, Apple Safari, Mozilla Firefox, and Microsoft Edge.

4.4 UI / Design of Application

- **<u>Part 1: Mobile Frontend (Flutter App)</u>**

Figure 4.2: myMove App Initialization & Splash Screen

Figure 4.2 shows the loading process and splash screen displayed when opening the app. The splash screen prepares the main application services, checks the user's saved login session, and then smoothly takes the driver to either the login page or the main dashboard based on their login status.

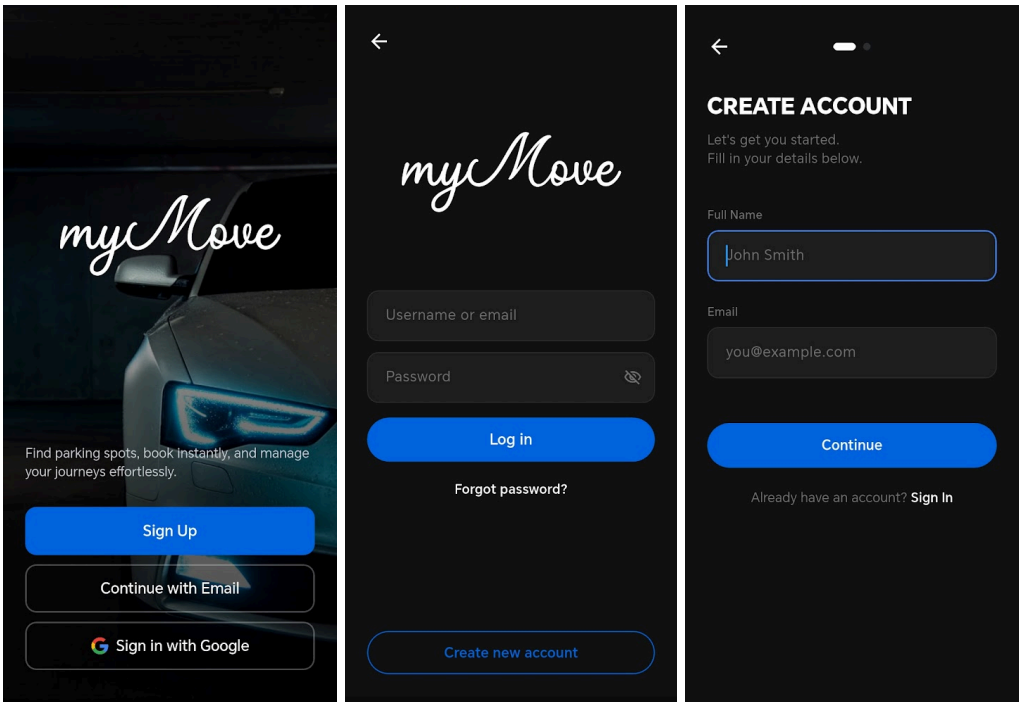


Figure 4.3: User Authentication (Login and Registration Screens)

Figure 4.3 shows the user authentication interface, including the login and new user registration. Users can log in using their registered email and password or sign in using Google OAuth. The registration form also checks the password strength while the user enters it and allows them to create their user profile.

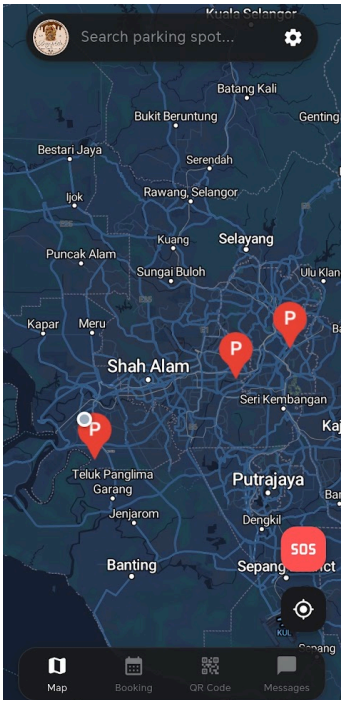


Figure 4.4: Main Dashboard & Parking Locations

Figure 4.4 displays the main user dashboard, which is a place for finding nearby parking facilities. It shows real-time available parking locations, search filters by name, quick navigation options, and summary cards for active bookings.

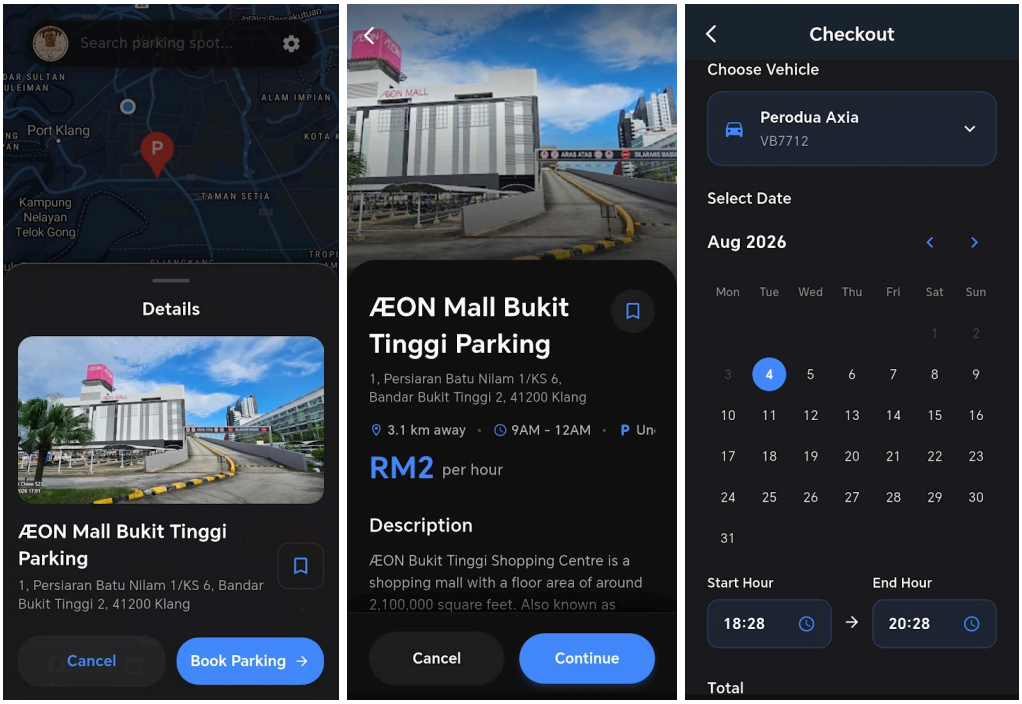


Figure 4.5: Parking Location Details, Pricing Structure and Select Date & Time

Figure 4.5 shows the facility details page, that shows the operating hours, hourly rates, available facilities, and parking slot availability. Users can choose their preferred parking date, start time, and reservation duration using an interactive date and time picker before selecting a parking spot.

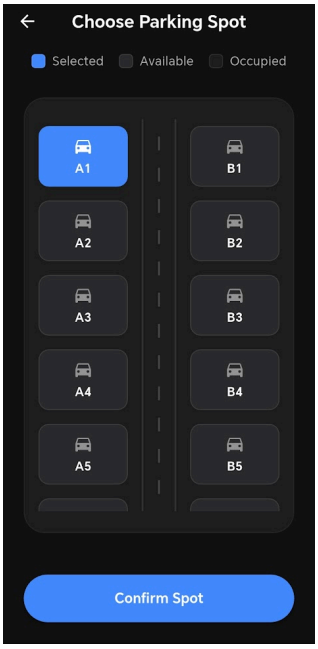


Figure 4.6: Real-time Parking Grid with Indicators

Figure 4.6 shows the real-time interactive parking layout. The interface shows parking spots using different colors based on their current status: dark grey for available spots, light grey for occupied spots updated in real time from the HC-SR04 sensors through Firebase RTDB, and blue for the parking spot currently selected by the user.

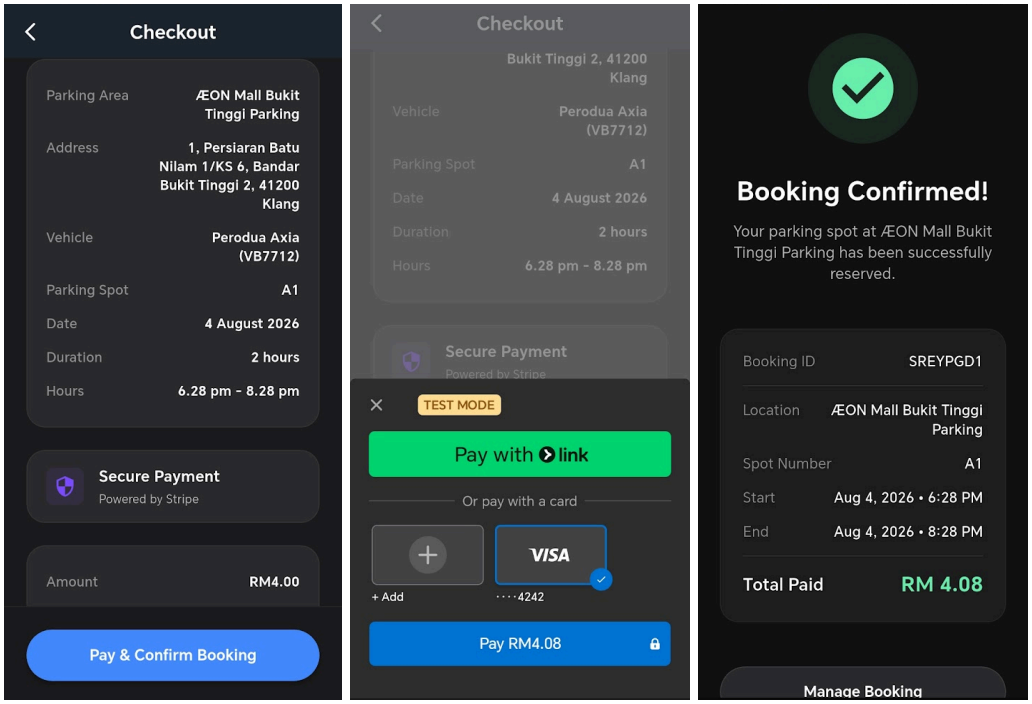


Figure 4.7: Booking Reservation Summary and Checkout Interface

Figure 4.7 shows the booking summary and checkout confirmation screen. It provides a payment breakdown, including the selected parking spot ID, booking duration, parking rate, taxes, and selected vehicle. The screen also connects directly to the Stripe SDK to allow users to securely authorize their card payment.

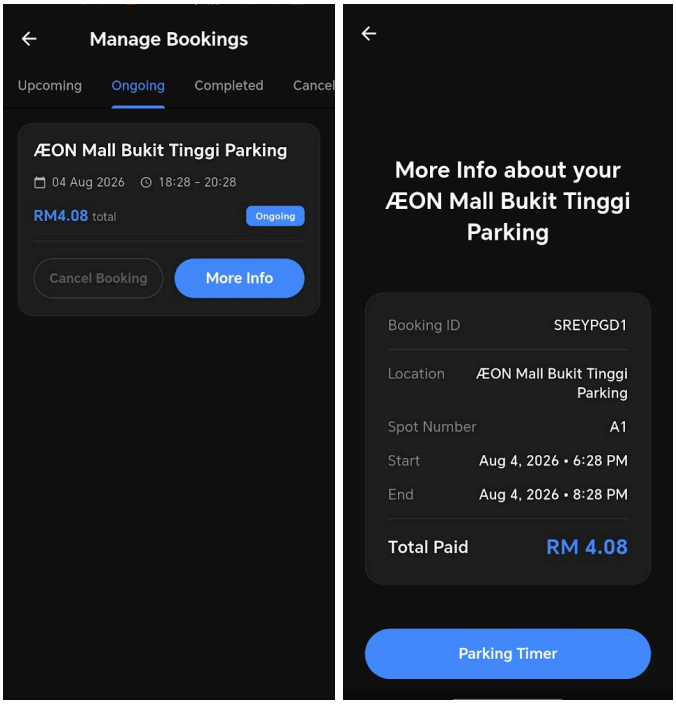


Figure 4.8: Manage Bookings and More Info Page

Figure 4.8 shows the active bookings management screen. Drivers can view their active, upcoming, and completed bookings, and check their digital entry pass details.

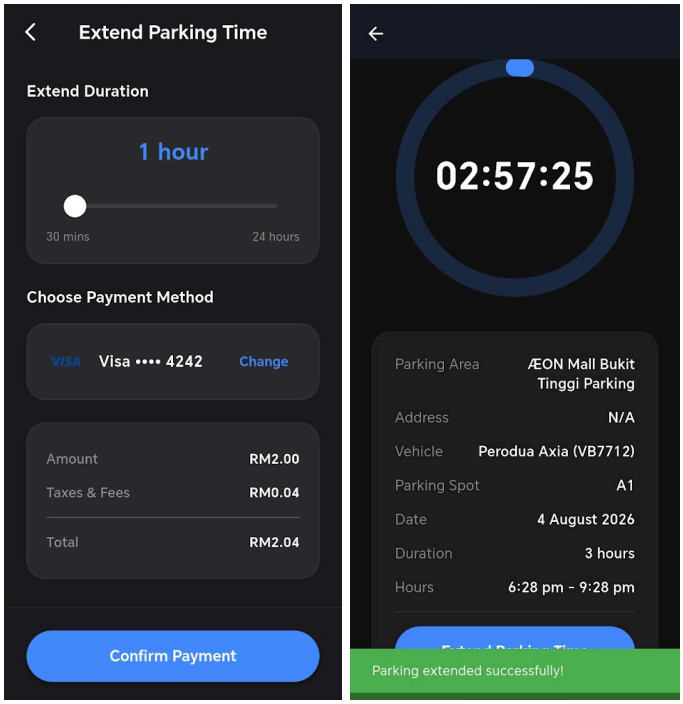


Figure 4.9: Extend Parking and Successful Page

Figure 4.9 shows the parking time extension and duration of the current parking. Drivers can extend their current parking session remotely by adding extra hours and making the additional payment without needing to return to their vehicle.

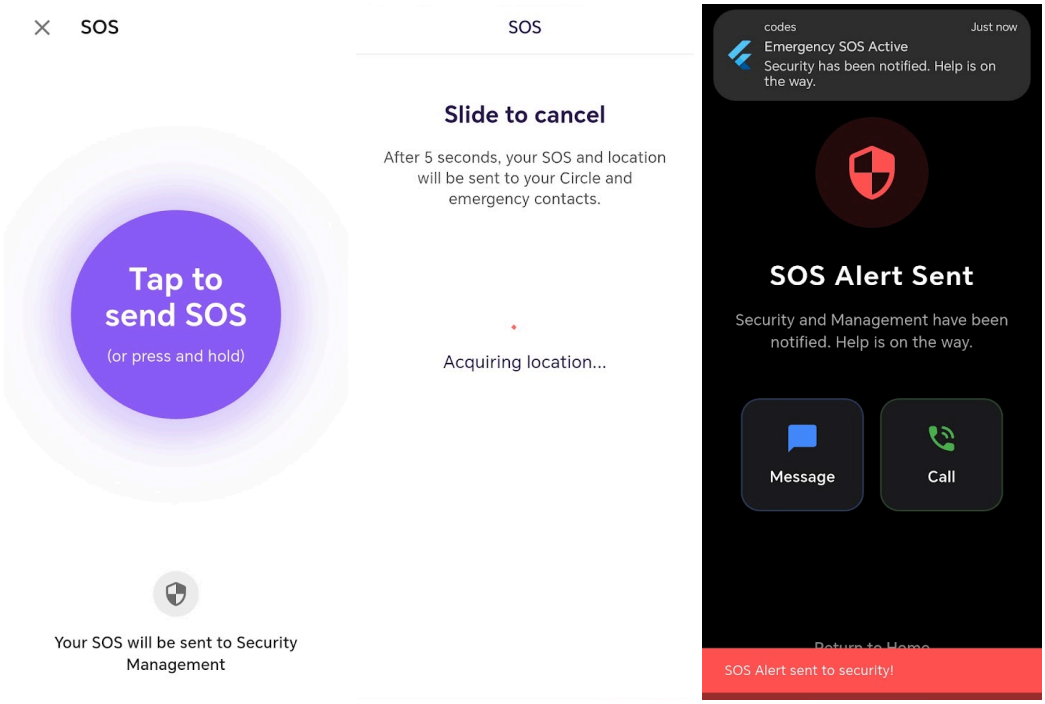


Figure 4.10: SOS / Emergency Panic Alert Trigger Overlay

Figure 4.10 shows the SOS emergency button and alert screen. When the emergency button is pressed, an urgent alert is immediately sent to Cloud Firestore, including the user's current location and contact details. The information is then sent to the Admin Dashboard for further assistance.

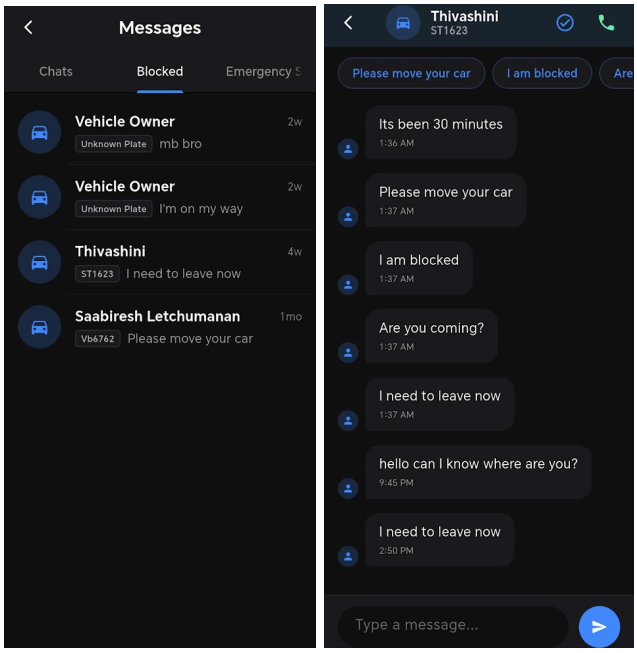


Figure 4.11: In-App Real-time Chat

Figure 4.11 shows the driver-to-driver messaging interface. This allows vehicle owners to send and receive real-time messages to move the vehicle during double-parking situations without sharing their personal phone numbers.

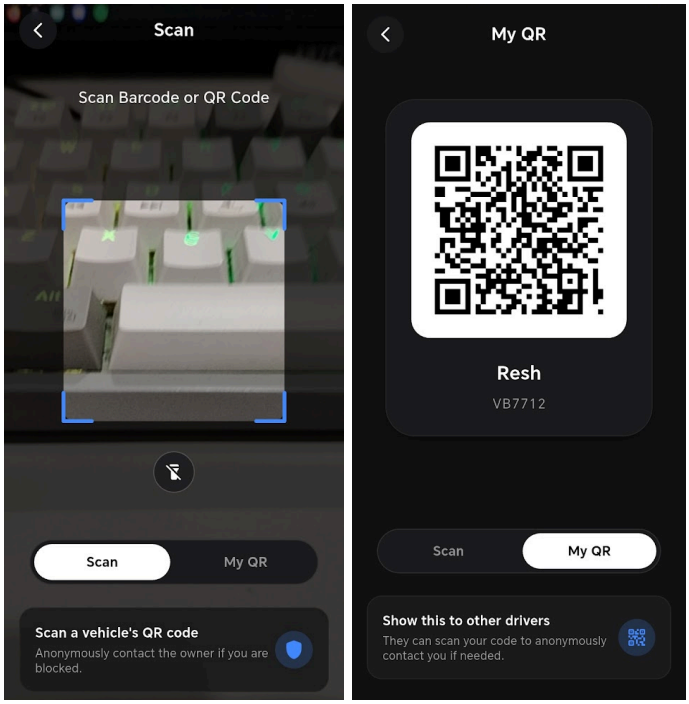


Figure 4.12: QR Code Scanner and Display for Vehicle Identification / Chat

Figure 4.12 shows the camera scanner interface and vehicle QR code display. Vehicle owners can show their unique QR code, while other drivers can use the in-app scanner to contact the vehicle owner anonymously.

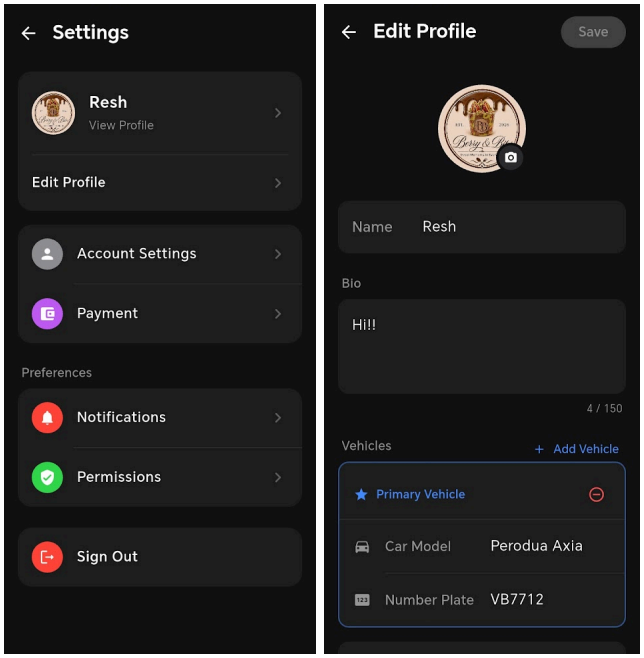


Figure 4.13: Main Settings & Edit Profile / Vehicle Page

Figure 4.13 presents the profile customization and other settings. Users can update their personal information, upload a profile photo from their device gallery, and manage multiple registered vehicles, including the vehicle make, model, plate number and more.

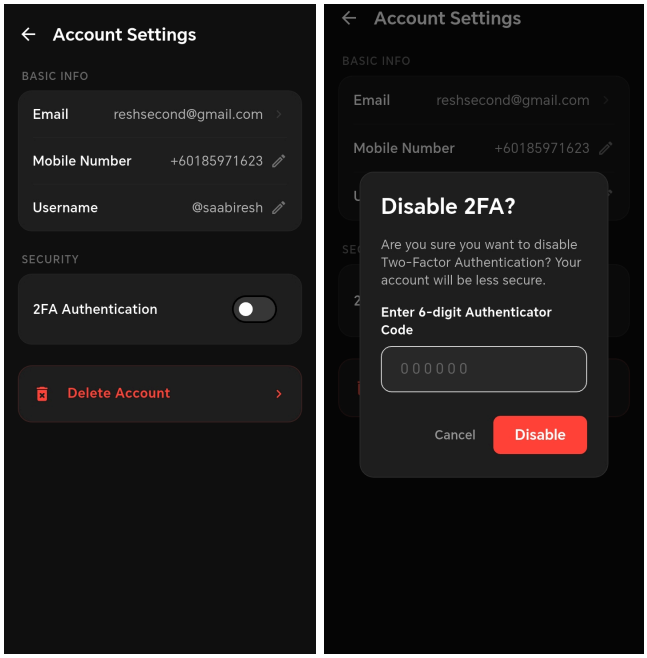


Figure 4.14: Account Settings and Two-Factor Authentication (2FA)

Figure 4.14 shows the security settings and Two-Factor Authentication (2FA) setup. Users can enable 2FA by scanning a generated QR code with an authenticator app, such as Google Authenticator, and then verifying the 6-digit security code.

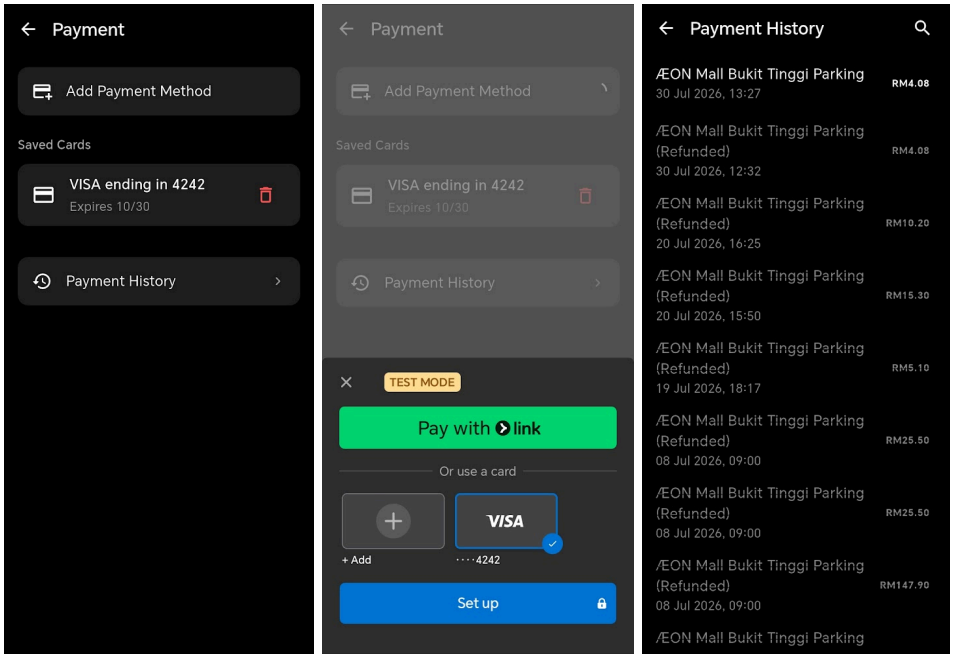


Figure 4.15: Payment Settings / Stripe Payment Method and Payment History

Figure 4.15 shows the saved payment methods and transaction history. Users can securely add or remove their credit or debit card details and view their previous receipts.

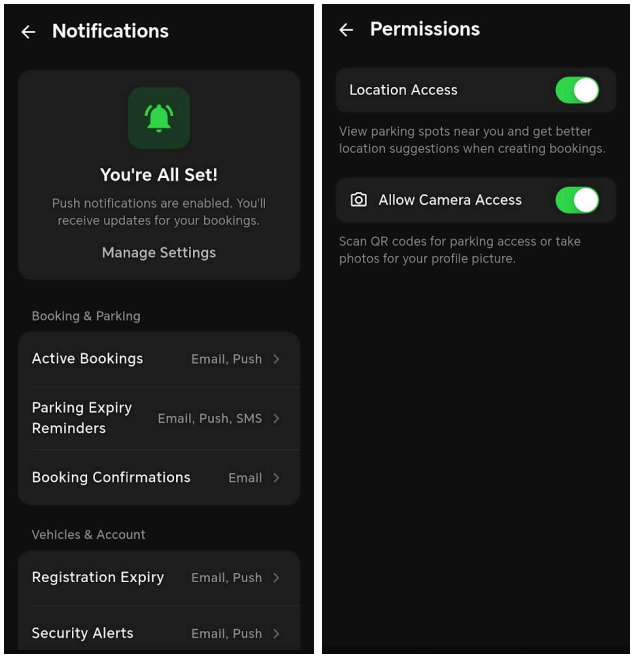


Figure 4.16: Push Notification Settings and Permissions Access Settings

Figure 4.16 shows the push notification settings. Drivers can turn on or off permissions for device location (to find nearby parking facilities) and camera access (for QR scanning), as well as set notifications for booking expiry and security alerts.

Flutter Code Snippets (Frontend)

```
void _listenToRealtimeSensors() {
 final dbRef = FirebaseDatabase.instance.ref(_rtdbPath);
  _rtdbSubscription = dbRef.onValue.listen((DatabaseEvent event) {
   if (!mounted) return;

   final data = event.snapshot.value;
   if (data is Map) {
     setState(() {
       data.forEach((key, value) {
         final uiSlot = _slotMapping[key.toString()];
         if (uiSlot != null) {
           if (value.toString() == 'occupied') {
             _sensorOccupiedSpots.add(uiSlot);
             if (_selectedSpot == uiSlot) {
               _selectedSpot = null; // Deselect if occupied by sensor
             }
           } else if (value.toString() == 'available') {
             _sensorOccupiedSpots.remove(uiSlot);
           }
         }
       });
     });
   }
 });
}
```

Table 4.1: Real-time RTDB Sensor Status Stream Listener

This snippet shows the stream listener that connects the Firebase Realtime Database (RTDB) sensor data to the Flutter UI and automatically updates the parking slot availability.

```dart
@override
 void initState() {
   super.initState();
   // Generate secure secret key on initialization
   _secret = TOTP.generateSecretKey();
 }
 @override
 void didChangeDependencies() {
   super.didChangeDependencies();
   final authProvider = Provider.of<AuthProvider>(context, listen: false);
   _qrUri = TOTP.getOtpAuthUrl(secret: _secret, email: authProvider.email);
 }
```

Table 4.2: Two-Factor Authentication (2FA) Setup & Secret Generation

Shows how the secret key is securely generated on screen initialization and mapped to standard TOTP **`otpauth://`** URI schema for compatibility with Google Authenticator.

```dart
final callable =
FirebaseFunctions.instance.httpsCallable('createSetupIntent');
     final response = await callable.call();
     final clientSecret = response.data['clientSecret'];
     final customerId = response.data['customer'];
     final ephemeralKey = response.data['ephemeralKey'];
     if (clientSecret == null) {
       throw Exception('Failed to get setup intent secret');
     }
     await Stripe.instance.initPaymentSheet(
       paymentSheetParameters: SetupPaymentSheetParameters(
         setupIntentClientSecret: clientSecret,
         customerId: customerId,
         customerEphemeralKeySecret: ephemeralKey,
         merchantDisplayName: 'myMove',
         style: ThemeMode.dark,
       ),
     );
     await Stripe.instance.presentPaymentSheet();
```

Table 4.3: Stripe SDK Setup Intent Card Addition Integration

Secure client-side Stripe Payment Sheet integration. It triggers a secure Firebase HTTPS Callable Cloud Function to retrieve Stripe Setup Intent credentials (clientSecret, customer, ephemeralKey) and initializes/displays the Stripe SDK modal checkout sheet.

```
StreamBuilder<QuerySnapshot>(
 stream: FirebaseFirestore.instance
     .collection('bookings')
     .where('userId', isEqualTo: user.uid)
     .snapshots(),
 builder: (context, snapshot) {
   // ... validation and loading checks ...
   final allPayments = snapshot.data!.docs.where((doc) {
     final data = doc.data() as Map<String, dynamic>;
     final docStatus = data['status'] as String? ?? 'pending';
     return docStatus == 'completed' || docStatus == 'active' || docStatus
== 'canceled';
   }).map((doc) {
     final data = doc.data() as Map<String, dynamic>;
     final isLegacyPrice = !data.containsKey('totalPaid');
     final double basePrice = (data['totalPrice'] ?? 0).toDouble();
     final double priceAmount = isLegacyPrice
         ? basePrice * 1.02
         : (data['totalPaid'] ?? 0).toDouble();

     final startStr = data['startDateTime'] as String?;
     final date = startStr != null
         ? DateFormat('dd MMM yyyy,
HH:mm').format(DateTime.parse(startStr).toLocal())
         : 'Unknown Date';
     // ... maps to payment list elements ...
   });
 }
)
```

Table 4.4: Querying User Booking Transactions from Cloud Firestore

Demonstrates the real-time StreamBuilder query that pulls user-specific bookings from Cloud Firestore and dynamically maps them to a transaction history model.

```
Future<void> _handlePermissionRequest(Permission permission, bool
isSwitchEnabled) async {
   final status = await permission.status;

   if (status.isPermanentlyDenied || (!isSwitchEnabled && status.isGranted))
{
     // If permanently denied, or user wants to revoke, they must go to
settings
     await openAppSettings();
   } else if (isSwitchEnabled && !status.isGranted) {
     final newStatus = await permission.request();
     if (newStatus.isPermanentlyDenied) {
       await openAppSettings();
     }
   }
   await _checkPermissions();
 }
```

Table 4.5: Dynamic System Permissions Handler

Coordinates OS-level permission requests using the permission_handler package, handling standard prompts and permanent denials by redirecting users to system settings as needed.

- **Part 2: Web Scanner (Guest Request Portal)**

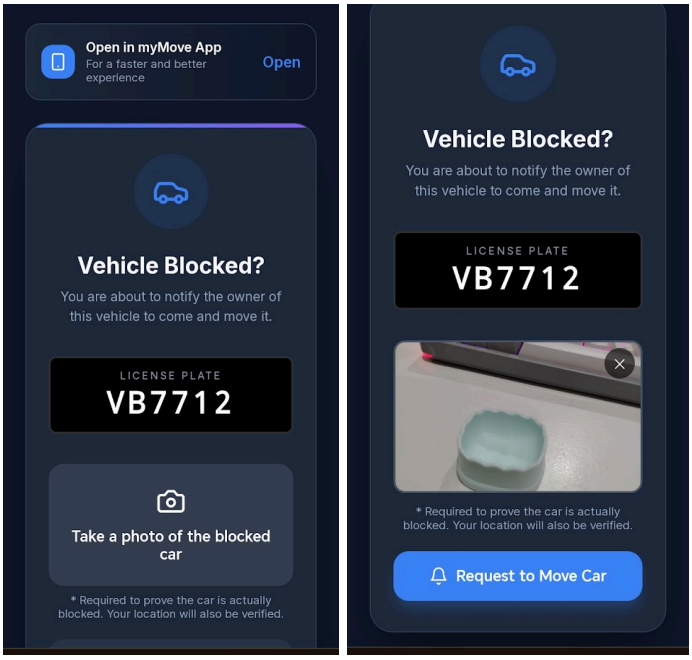


Figure 4.17: Webpage for Blocked Vehicle QR Scan and Photo Proof Verification

Figure 4.17 displays the lightweight, zero-install Scanner-Web guest portal loaded in a mobile browser upon scanning a vehicle's windshield QR code. Guest drivers can capture and upload photo proof of a double-parking blockage and submit a move request anonymously.

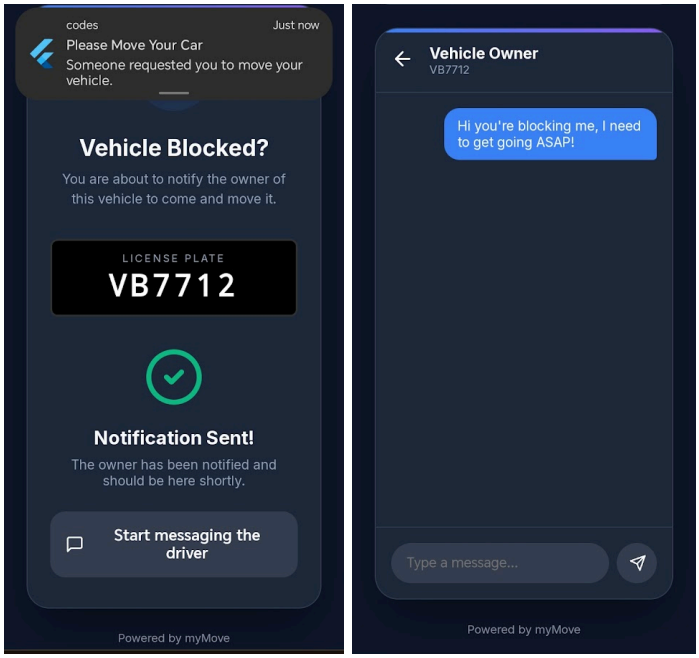


Figure 4.18: Successful Request Notification and Web-Based Chat

Figure 4.18 shows the notification confirmation screen and guest web messaging view. After the request is submitted, the system sends an instant push notification to the vehicle owner and creates a temporary anonymous web chat session.

Web Scanner Code Snippets (Frontend - Typescript)

```
const handleRequestMove = async () => {
 if (!targetId || requestStatus === 'sending' || !user || !photoFile)
return;
 setRequestStatus('sending');
 try {
   // 1. Fetch Guest Geolocation
   const location = await getLocation();

   // 2. Upload verification photo to Cloud Storage
   const photoRef = ref(storage,
`move_requests/${user.uid}_${Date.now()}.jpg`);
   const metadata = { contentType: photoFile.type || 'image/jpeg' };
   await uploadBytes(photoRef, photoFile, metadata);
   const photoUrl = await getDownloadURL(photoRef);

   // 3. Trigger Firebase HTTPS Callable Cloud Function
   const requestWebMoveCar = httpsCallable(functions, 'requestWebMoveCar');
   await requestWebMoveCar({
     targetUserId: targetId,
     photoUrl,
     location
   });
   setRequestStatus('success');
 } catch (err: any) {
   console.error('Error requesting move:', err);
   setRequestStatus('error');
   setError(err.code === 'functions/resource-exhausted'
     ? 'Please wait a moment before sending another notification to prevent
spam.'
     : 'Failed to send notification.');
 }
};
```

Table 4.6: Guest Move Request Submission Logic

Shows the client-side process by checking the input details, getting the user's location, uploading the verification image to Firebase Storage, and running the secure Cloud Function.

- **Part 3: Admin Web Dashboard (Vite + React)**

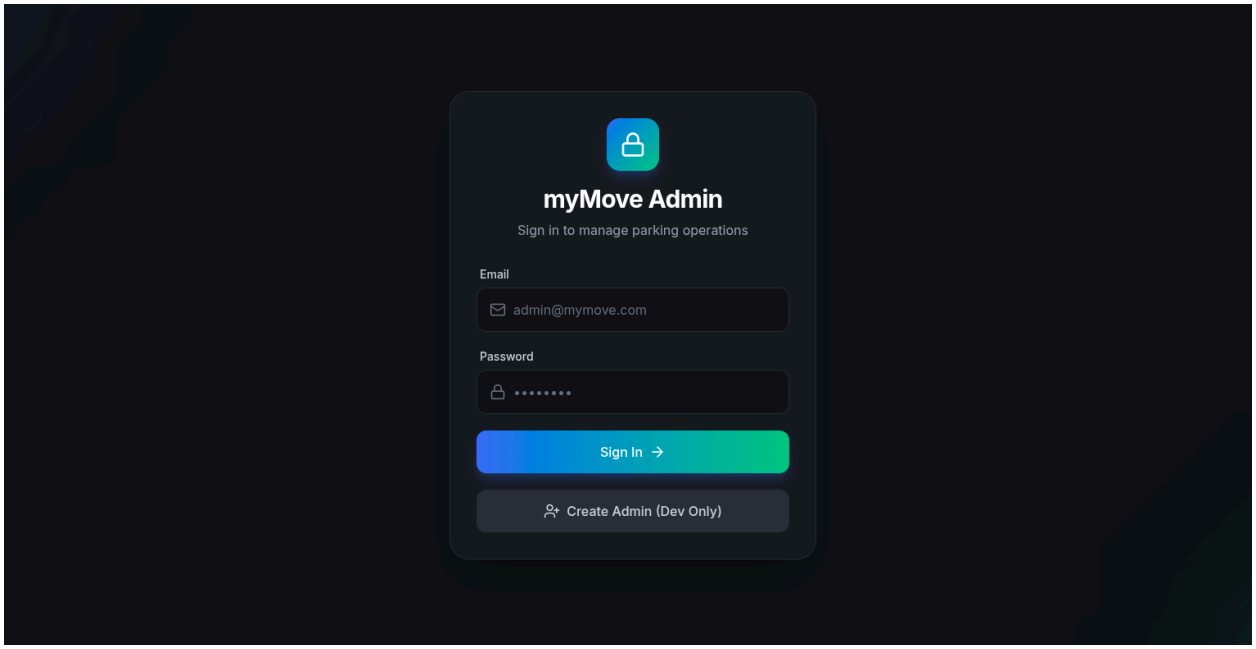


Figure 4.19: Admin Panel Login

Figure 4.19 presents the secure admin web portal login screen. Parking management and security staff can log in using their credentials to access management features.

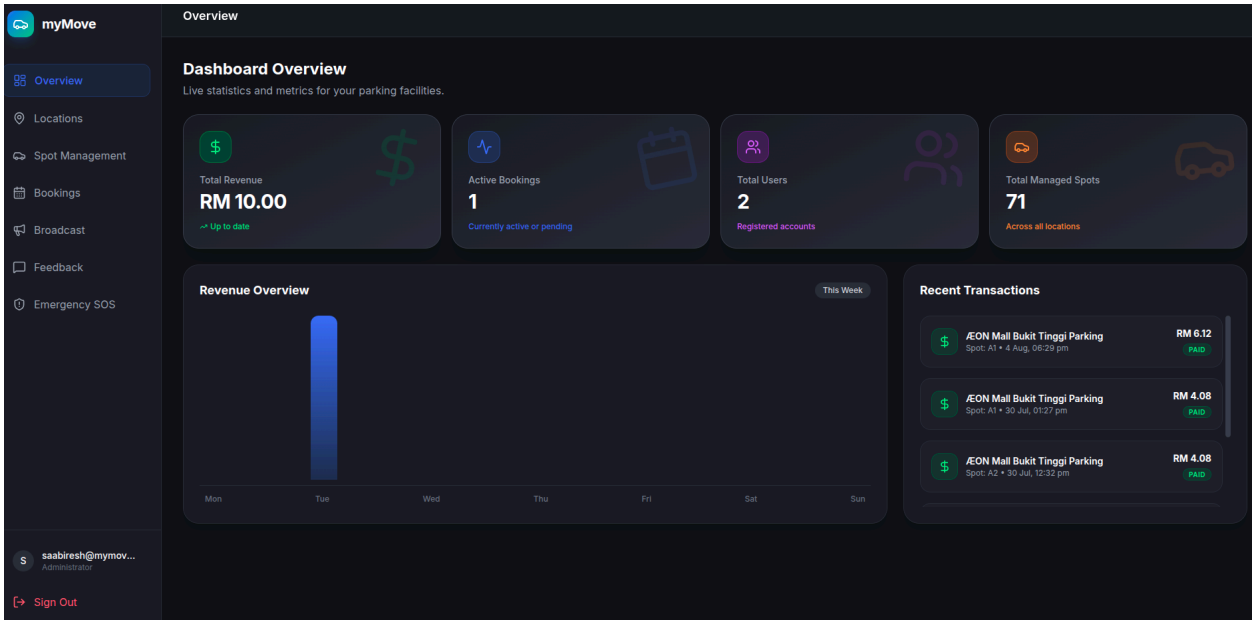


Figure 4.20: Admin Dashboard with Analytics and Weekly Revenue

Figure 4.20 shows the main admin dashboard with real-time parking availability information, active emergency alerts, weekly revenue charts and more.

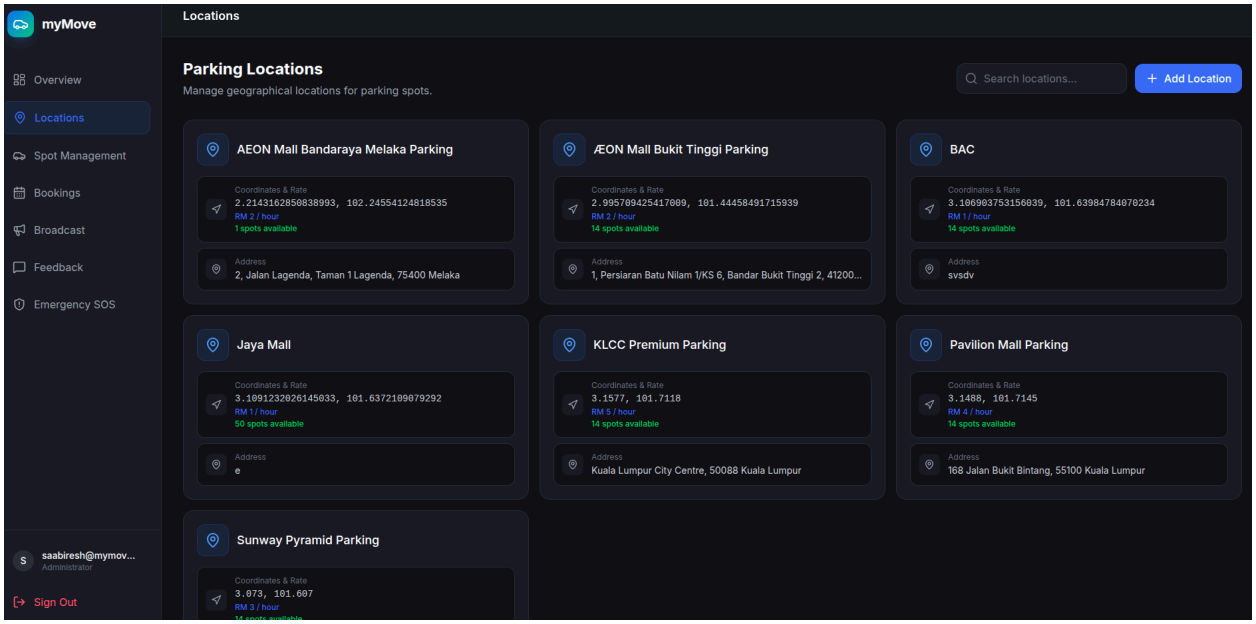


Figure 4.21: Parking Location Management Panel

Figure 4.21 shows the parking location management interface. Administrators can add, edit, or remove parking locations, manage the total number of parking slots, and set the hourly parking rates.

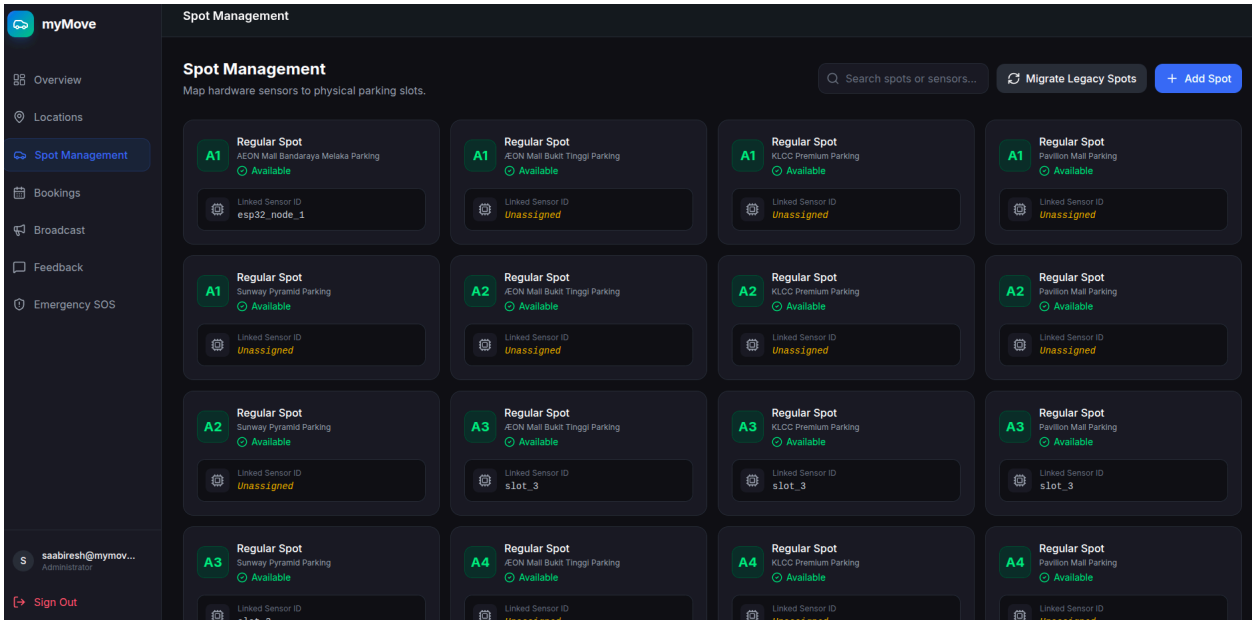


Figure 4.22: Manage Parking Spot

Figure 4.22 shows the parking spot mapping interface. Administrators can check the status of individual parking spots and connect hardware sensor IDs to specific parking bays.

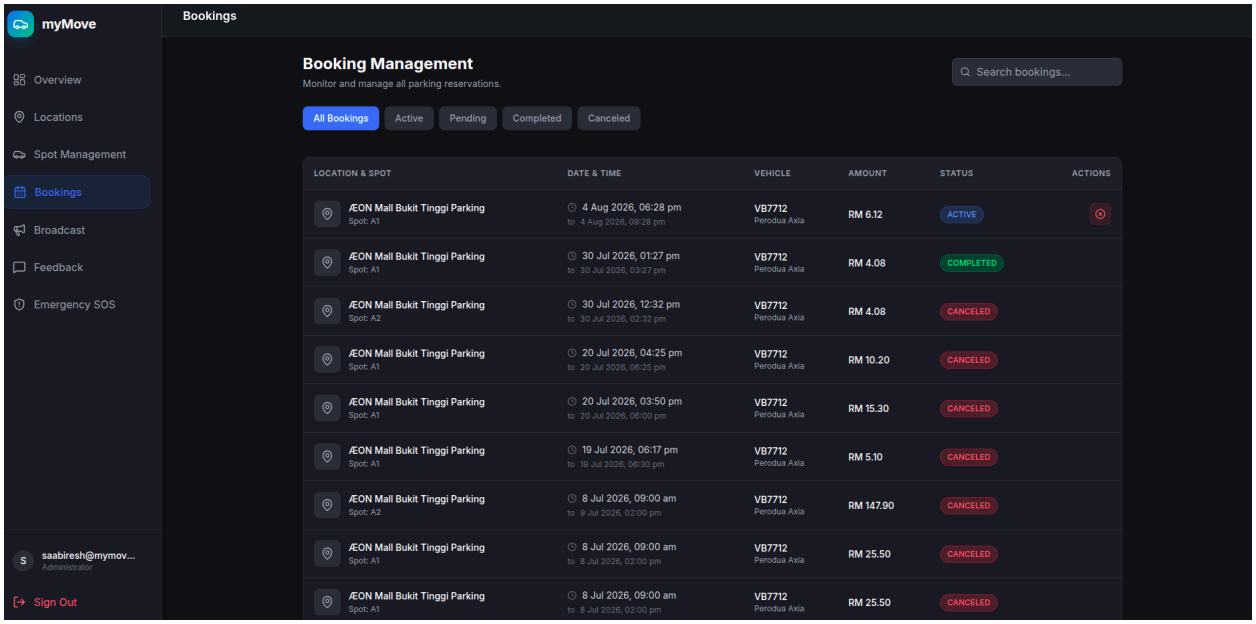


Figure 4.23: Booking Management Monitor

Figure 4.23 shows the real-time booking management on the admin dashboard. Admin can view, filter, and track all active, upcoming, and completed user bookings across parking facilities, and view detailed booking records.

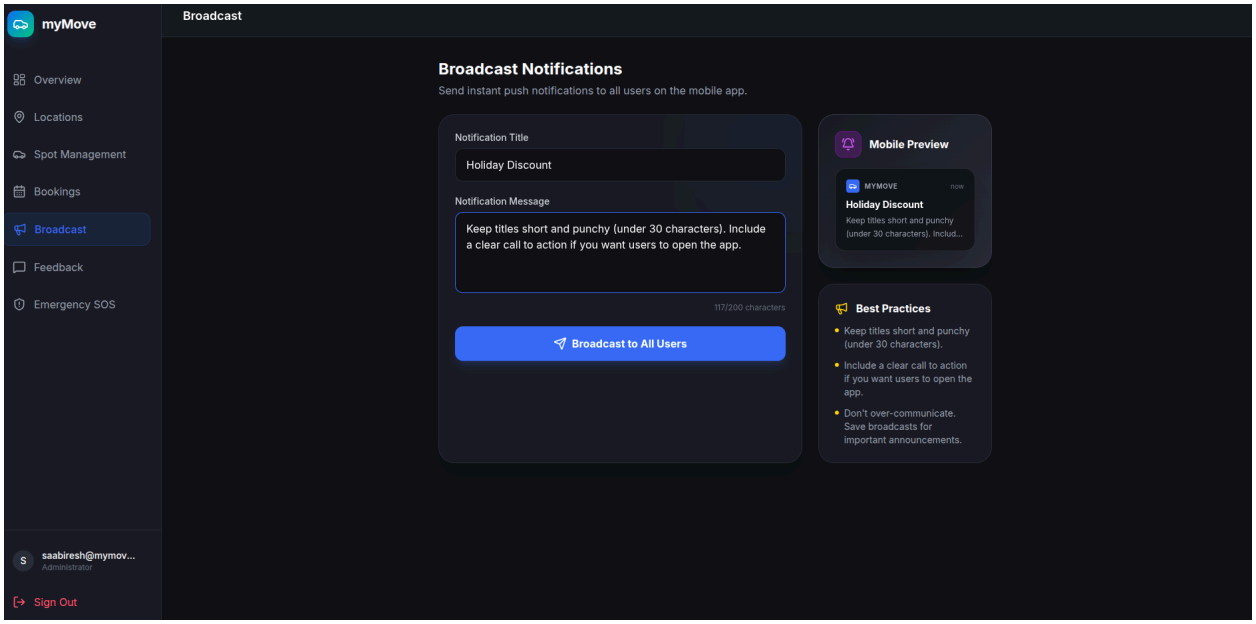


Figure 4.24: Broadcast Notification to All Users

Figure 4.24 shows the announcement page. Admin can create and send push notifications to all registered mobile app users about maintenance schedules or important security alerts.

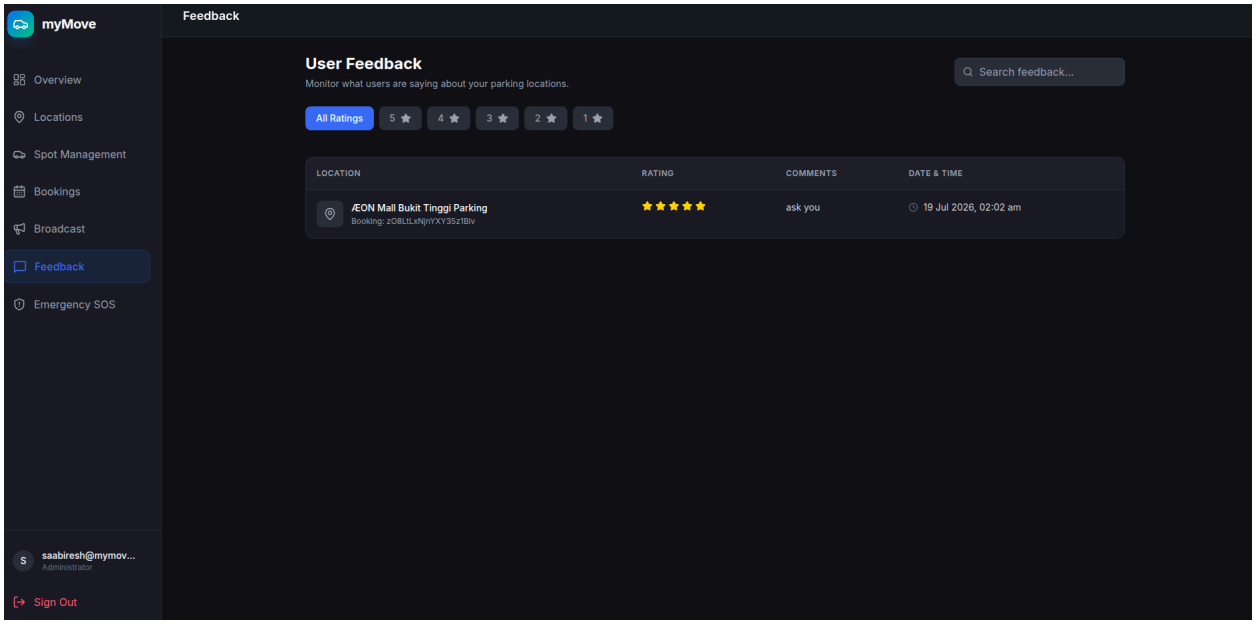


Figure 4.25: User Acceptance Testing Feedback Page

Figure 4.25 shows the feedback page. This page collects user survey responses, rating results, and written feedback to rate the system's usability, driver satisfaction, and usefulness of the app features.

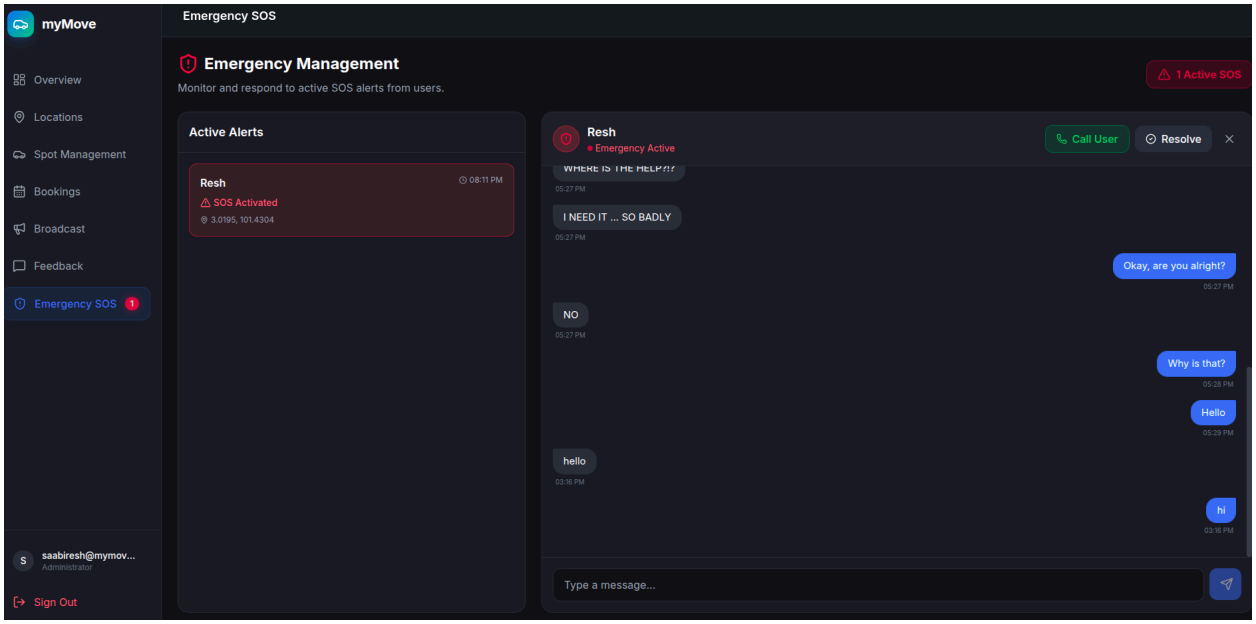


Figure 4.26: SOS / Emergency Panic Trigger Alarm

Figure 4.26 displays the live SOS emergency alert screen. When a driver activates an emergency, the dashboard immediately shows a pop-up window with the user's vehicle details, contact information, and exact GPS location for quick security response.

Admin-Web Code Snippets

Real-time Active Booking Calculations & Listeners

```
bookings.forEach(booking => {
 if (booking.status !== 'canceled') {
   totalRevenue += (booking.totalPrice || booking.totalPaid || 0);
 }
  // Real-time effective active status check
 const now = new Date();
 const endDateTime = booking.endDateTime ? new Date(booking.endDateTime) :
null;
 let isEffectiveActive = false;
  if (booking.status === 'pending') {
   isEffectiveActive = true;
 } else if (booking.status === 'active') {
   if (!endDateTime || endDateTime >= now) {
     isEffectiveActive = true;
   }
 }
  if (isEffectiveActive) {
   activeBookings++;
 }
});
```

Table 4.7: Real-Time Active Booking Calculations & Listeners

Collects booking data in real time and checks whether each booking is still "Active" by comparing the **endDateTime** with the device's current time.

4.5 Backend Implementation

4.5.1 Backend Architecture

myMove uses a serverless Backend-as-a-Service (BaaS) architecture by Google Firebase. This means it does not require a physical or virtual server to manage. Instead, the Flutter mobile app and React/Vite admin web portal connect directly and securely to Firebase cloud services through authenticated SDKs.

This architecture shows real-time sensor updates delivered in less than one second, makes user login and account management easier, and can automatically support hundreds of drivers and admin using the system at the same time.

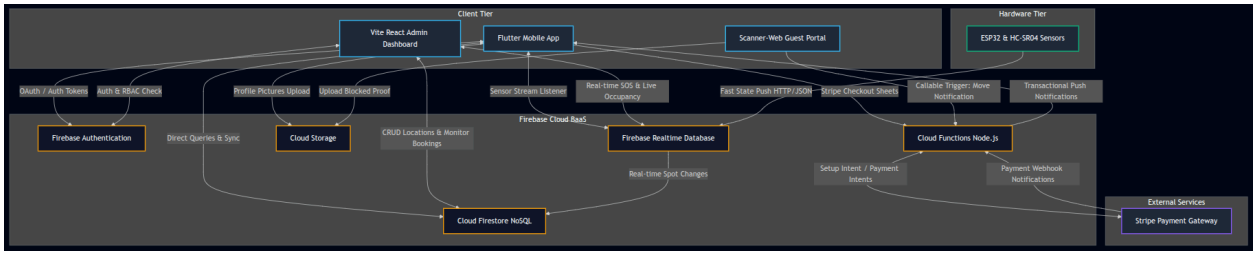


Figure 4.27: myMove Architecture Diagram

The system is divided into four main components:

1. **<u>Client Tier:</u>**

- **Flutter Mobile App:** The UI is written in Dart which is provided for drivers. It connects to Firebase services using the official **firebase_core**, **firebase_auth**, **cloud_firestore**, **firebase_database**, and **firebase_messaging plugins**.
- **React Web Admin Dashboard:** The admin page is built with React and Vite. It allows parking management to review reports, view revenue data, manage parking locations, and monitor active SOS alerts.
- **Scanner-Web Portal:** A React webpage made for guests. When a guest scans the QR code attached to a vehicle blocking their way, the website opens directly without requiring an app download.

2. **Firebase Cloud BaaS:**

- **Authentication:** Manage user registration and login using JSON Web Tokens (JWT) which supports standard email and password authentication, linking phone numbers, and maintaining user sessions.
- **Cloud Firestore:** A flexible NoSQL database that stores all important system data, such as user profiles, booking records, active chat sessions, and parking locations.
- **Firebase Realtime Database (RTDB):** A fast WebSocket-based JSON database which is designed to quickly send the status (available or occupied) of parking spots from the hardware directly to connected users within milliseconds.
- **Cloud Storage**: Serves as a storage system for files, handling driver verification photos for blocked parking requests and user-uploaded profile pictures.
- **Cloud Functions:** A Node.js backend that performs secure tasks that should not be handled by client-side code, such as limiting guest requests, and checking for overlapping active bookings.

3. **Hardware Tier (IoT System):**

- **ESP32 Microcontroller:** Connected to HC-SR04 sensors to monitor parking slots. It runs firmware that verifies whether each parking slot is occupied and sends status updates directly to the Firebase RTDB.

4. **External Infrastructure:**

- **Stripe Payment Gateway:** Handles payment card and processing. The client requests a session through Cloud Functions, which provides a setup token that allows the Android SDK to display the payment form and reducing to store sensitive info locally.

4.6 Database Design

The system uses Cloud Firestore and Firebase Realtime Database (RTDB) to manage the data. Cloud Firestore is used to store information, while Firebase Realtime Database (RTDB) is used to handle real-time parking sensor updates. Since Firestore is a document-based database, information is organized into collections and documents. Firestore Security Rules are used to control access to the database and make sure that only authorized users can view or change information.

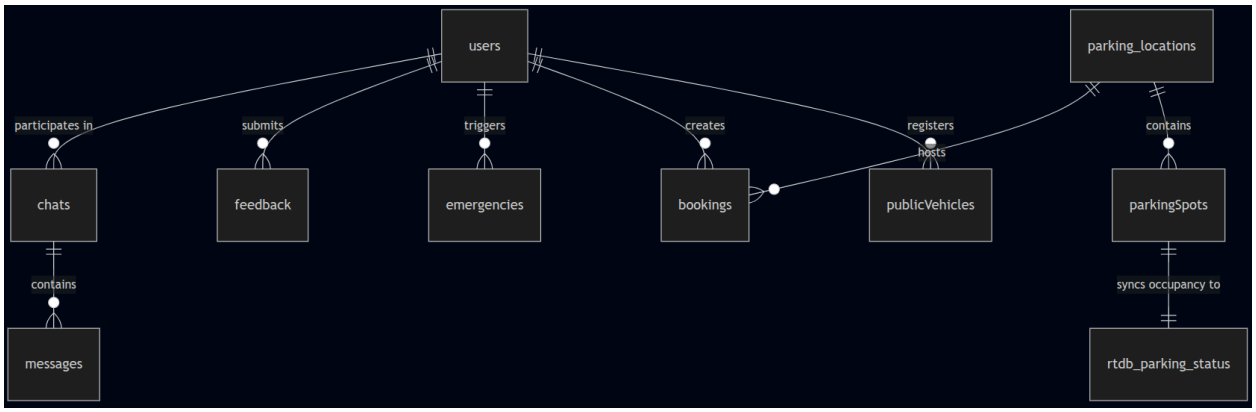


Figure 4.28: ER Diagram of Cloud Firestore Database Schema

4.6.1 Cloud Firestore Collections Schema

The database is organized the following main collections:

Field Name	Data Type	Description
uid (Document ID)	String	Unique identifier linked to Firebase Auth.
displayName	String	User's full name
email	String	System access tier: **user** or **admin**
twoFactorSecret	String	Base32 secret key used for Google Authenticator TOTP authentication.
isTwoFactorEnabled	Boolean	True if Two-Factor Authentication is active.
fcmToken	String	Firebase Cloud Messaging token for push notifications.
vehicles	Array (Map)	Nested list of user vehicles: [{**brand**, **model, plateNumber**, **isPrimary**}].
createdAt	Timestamp	Timestamp indicating account registration date.

Table 4.8: Cloud Firestore users Collection Schema

Stores user information, roles, notification, and security that is required for Multi-Factor Authentication (MFA).

Field Name	Data Type	Description
vehicleId (Document ID)	String	Matches the registered owner's UID
ownerID	String	References the users UID
plateNumber	String	License plate number (indexed for fast search)
brand	String	Vehicle manufacturer brand.
model	String	Specific vehicle model name.
color	String	Vehicle color description
isActive	Boolean	Active flag indicating if the vehicle is currently in use.
createdAt	Timestamp	Registration timestamp

Table 4.9: Cloud Firestore publicVehicles Collection Schema

A registered vehicle stores license plate numbers and vehicle brands with owner IDs to allow quick QR code searches.

Field Name	Data Type	Description
locationId (Document ID)	String	Unique parking location identifier.
name	String	Human-readable building name (e.g. Main Block)
address	String	Full physical street address.
latitude	String	GPS latitude coordinate
longitude	String	GPS longitude coordinate
hourlyRate	String	Cost per hour of parking (in MYR)
operatingHours	Boolean	Opening and closing rules (e.g. "08:00 - 22:00")

Table 4.10: Cloud Firestore parking_locations Collection Schema

Contains information about parking facilities, including parking facility and pricing rates.

Field Name	Data Type	Description
spotId (Document ID)	String	Composite key combining location and spot name.
locationId	String	Foreign Key referencing **parking_locations.locationId**
spotName	String	User display code (e.g., "A1", "A2").
hardwareSensorId	String	Unique hardware sensor ID printed on the physical sensor.
createdAt	Timestamp	Creation date of mapping.

Table 4.11: Cloud Firestore parkingSpots Collection Schema

Connects parking spots with their physical hardware sensor units.

Field Name	Data Type	Description
bookingId (Document ID)	String	A unique and secure ID created for each transaction.
userId	String	Foreign Key referencing the driver's **users.uid**
locationId	String	Foreign Key referencing **parking_locations.locationId.**
spotName	String	The reserved parking spot code.
startDateTime	String (ISO)	Reservation start boundary.
endDateTime	String (ISO)	Reservation expiration boundary.
status	String	Booking status: **pending, active, completed, canceled**
totalPrice	Number	Initial calculated rate.
totalPaid	Number	Final actual payment captured (with Stripe processing fees).
paymentIntentId	String	Reference ID mapping to the payment in the Stripe Dashboard.
createdAt	Timestamp	Creation timestamp.

Table 4.12: Cloud Firestore bookings Collection Schema

Stores parking records, including duration, payment information, and status updates.

Field Name	Data Type	Description
chatId (Document ID)	String	A unique and secure ID created for each transaction.
participants	String	Foreign Key referencing the driver's **users.uid**
lastMessage	String	Foreign Key referencing **parking_locations.locationId.**
lastMessageAt	String	The reserved parking spot code.
blockedDriverId	String (ISO)	Reservation start boundary.
vehicleOwnerId	String (ISO)	Reservation expiration boundary.

Table 4.13: Cloud Firestore chats & messages (Subcollection) Schema

Stores active real-time chat sessions and peer-to-peer messages between drivers during double-parking coordination.

Field Name	Data Type	Description
messageId (Document ID)	String	Unique message ID
senderId	String	UID of the message sender
receiverId	String	UID of the message receiver.
messageText	String	Decrypted message content
isRead	Boolean	True if the receiver has viewed the message
createdAt	Timestamp	Timestamp representing exact delivery time

Table 4.14: Cloud Firestore chats Collection & messages Subcollection Schema

Field Name	Data Type	Description
emergencyId (Document ID)	String	Unique alert ID
userId	String	Foreign Key referencing the reporting **users.uid**.
locationAddress	String	Nearest street name fetched via geolocation APIs.
latitude	Number	Exact trigger latitude
longitude	Number	Exact trigger longitude
status	String	Management status: **active** or **resolved**
createdAt	Timestamp	Timestamp of alert trigger

Table 4.15: Cloud Firestore emergencies Collection Schema

Emergency dashboard that tracks panic alerts triggered by mobile users.

Field Name	Data Type	Description
`feedbackId (Document ID)`	String	Unique feedback ID
`userId`	String	Foreign Key referencing **users.uid**
`score`	Number	Satisfaction rating from 1 to 5
`comments`	String	Open-ended user remarks
`createdAt`	Timestamp	Submission timestamp

Table 4.16: Cloud Firestore feedback Collection Schema

Collects user feedback with comments during the testing process.

4.6.2 Firebase Realtime Database (RTDB) Schema

The Realtime Database uses a simple JSON form that allows fast data reading and writing by low-power IoT microcontrollers. Data is organized using location and parking spot keys to reduce the amount of data transferred and improve system efficiency.

```
{
  "parking_status": {
    "location_A": {
      "slot_1": "occupied",
      "slot_2": "available",
      "slot_3": "occupied"
    },
    "location_B": {
      "slot_1": "available",
      "slot_2": "available"
    }
  }
}
```

Table 4.17: Firebase Realtime Database (RTDB) Schema

- Data Path: **/parking_status/{locationId}/{spotName}**
- Payload values:
    - **"available":** The HC-SR04 sensor is above the set limit (> 10.0 cm), meaning the parking spot is empty.
    - **"occupied":** The sensor is at or below the set limit (≤ 10.0 cm), meaning a vehicle is present.

4.7 Summary

The implementation phase successfully turned the system requirements and design into a fully functional, secure, and connected IoT-based smart parking system. By using Flutter for the cross-platform mobile application, ESP32 microcontrollers with HC-SR04 ultrasonic sensors for real-time parking spot monitoring, and Firebase services (Cloud Firestore and Realtime Database) as the backend, myMove provides live parking updates with reliable data management. The Stripe SDK provides secure payment processing, while security features such as Two-Factor Authentication (2FA) and QR-based contact verification improve system security and user convenience. Firebase Cloud Functions handle the serverless business logic, while the React/Vite Web Admin Dashboard allows administrators to monitor and manage the system effectively. Overall, the completed myMove application meets the project objectives by providing a scalable, maintainable, and more effective solution compared to traditional parking management and vehicle blocking methods.

CHAPTER 5

TESTING & EVALUATION

5.1 Introduction

This chapter shows the testing results for the myMove. After completing the system development, different types of testing were carried out, including unit testing, integration testing, system testing, and User Acceptance Testing (UAT). The main purpose of these tests was to make sure that the hardware, mobile application, guest web, and backend system work properly, securely, and within an acceptable response. Testing was carried out using a Android smartphones to check whether the system works properly across different environments and remains stable during normal use. The results presented in this chapter show that the myMove system meets the functional and performance requirements set for the project.

5.2 Project Deliverables

The development phase shows a complete set of software and hardware that work together to bring the myMove system:

1. Android mobile app with user login, real-time parking availability, parking bookings, payments, profile and vehicle management, 2FA security, permissions, and driver chat.
2. A web page created using React, TypeScript, and Vite for parking lot operators or security. It provides location and parking slot management, live occupancy monitoring, emergency/SOS alerts, and revenue charts.
3. A web designed for mobile browsers that opens when a guest scans the QR code on a double-parked vehicle. It allows guests to notify vehicle owners about double-parking situations without installing the mobile app.
4. A 3D prototype with ESP32, three ultrasonic sensors, and custom firmware to provide real-time parking updates.
5. A cloud-based backend using Firebase for user authentication, data storage, real-time sensor updates, payment processing, 2FA, and notifications.
6. A secure private repository organized into separate modules for the mobile app, web admin portal, guest web scanner, Firebase Cloud Functions, and ESP32 microcontroller firmware.
7. Project diagrams, database designs, requirement documents, and setup guides stored in the project documentation.

5.3 Software & Hardware Testing

Testing was carried out step by step to make sure the application is stable, and easy to use. Four main types of testing were performed which is, unit testing, integration testing, system testing, and usability testing.

5.3.1 Unit Testing

Unit testing was conducted to make sure that each function, user interface, and system logic worked correctly. Dart testing tools were also used for these tests, while manual tests were also performed.

Key unit test categories included:

- **User Authentication:** Checking email formats, checking password strength, and making sure the 2FA codes are generated and verified correctly.
- **IoT Occupancy:** Testing the distance logic where a distance below 10.0 cm is shown as "occupied", and a distance above 10.0 cm is shown as "available".
- **Data Validation:** Checking vehicle license plate details, making sure payment amounts are formatted correctly, and reading Firestore timestamps.
- **URL Construction:** Checking that the QR code generator creates the correct web URLs with the encrypted vehicle and owner IDs for double-parking.

A total of 45 unit tests were carried out, with a pass rate of 97.8%. Any issues found during testing were fixed immediately. A summary of the unit testing results is shown in Table 5.1:

Test Category	Number of Tests	Passed	Failed	Pass Rate
Authentication & 2FA	15	15	0	100%
IoT Distance Logic	10	10	0	100%
Input Form Validation	12	11	1	91.7%
QR Code & URL Utility	8	8	0	100%
Total	**45**	**44**	**1**	**97.8%**

Table 5.1: Unit Testing Summary

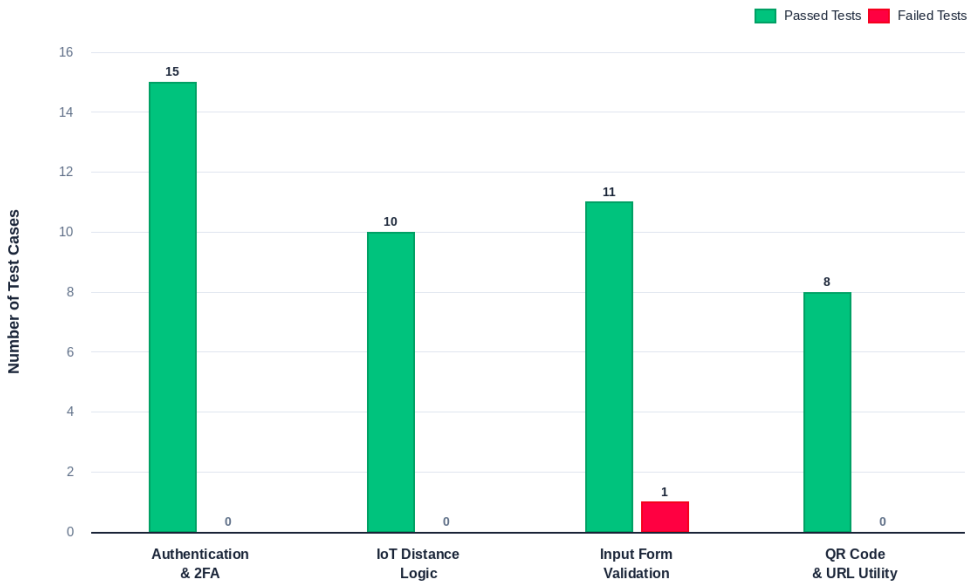


Figure 5.1: Unit Testing Execution Summary by Category

5.3.2 Integration Testing

Integration testing was used to check whether parts of the system could communicate and work together properly. The testing focused on the connections between the frontend applications, IoT hardware, and Firebase backend services.

Key integration scenarios tested:

1. **IoT Sensor-to-App Data Flow:** Checking that ESP32 sends an HTTP PUT request when the sensor detects a change in parking status. The updated status is then saved in Firebase RTDB and sent to the Flutter app to update the parking spot.
2. **Stripe Payment System:** Checks that starting the payment process calls the backend Cloud Function to request a Setup Intent from Stripe. The required information is then returned to the app so the payment form can be displayed.
3. **Double-Parking QR Contact Flow:** Checks that scanning a vehicle's QR code opens the scanner web page. When the guest selects "Notify Owner", the Firebase Cloud Function sends a notification to the vehicle owner, which appears as an alert in the app.
4. **SOS Emergency Alert:** Checks that pressing the SOS button in the app saves an emergency alert in Firestore. The admin dashboard receives the update immediately and displays an alert with a notification.

Integration Path	Test Cases	Passed	Failed	Results
IoT ESP32 → RTDB → Flutter Grid	10	10	0	Spot color updates within 1.5 seconds.
Stripe Sheet → Cloud Functions → Firestore	8	8	0	Tokenization and card mapping complete successfully.
Web-Scanner → Cloud Functions → FCM → Mobile	12	12	0	The owner receives an anonymous alert successfully.
Flutter SOS button → Firestore → Web Admin	8	8	0	Alarm rings and shows an active location on the admin map.
Total	**35**	**38**	**0**	**100%**

Table 5.2: Integration Testing Summary

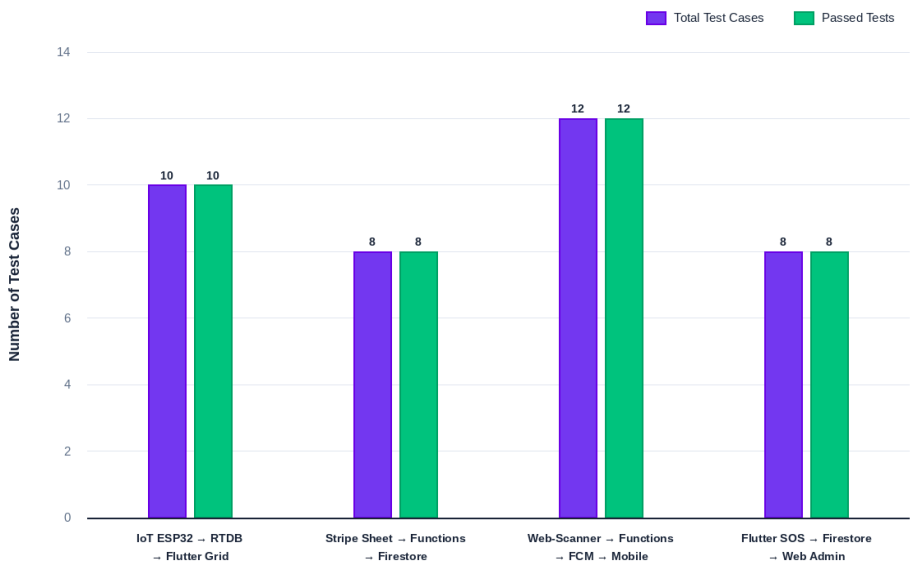


Figure 5.2: Integration Testing Execution Summary by Path

5.3.3 System Testing

System testing was carried out to check the complete user tour from start to finish under normal conditions. The main focus was on checking the system's response time, delay, and stability when running on different physical devices.

The system tests focused on four critical paths:

1. **End-to-End Reservation & Payment Cycle:** Test the full process of searching for a parking facility, select an available spot, making a booking, paying through Stripe, and confirm that the parking spot is successfully reserved.
2. **Sensor Occupancy Sync Speed:** Estimate time taken from placing an object in front of the HC-SR04 sensor until the parking spot changes to light grey in the mobile app.
3. **Double-Parking Alert Delivery Latency:** Estimate time between a guest pressing the "**Notify Owner**" button on the mobile web browser and the vehicle owner receiving the push notification.
4. **SOS Broadcast Speed:** Estimate time taken for the emergency alert card to appear on the admin dashboard after a user activates the SOS button in the app.

System performance test results are shown in Table 5.3:

Test Scenario	Devices Tested	Success Rate	Response Time
Complete Reservation & Payment	Android, React Web	100%	4.2 seconds
IoT Spot Occupancy Update	ESP32, Firebase RTDB, Android	100%	1.1 seconds
Double-Parking QR Notification	Android (Browser)	100%	1.9 seconds
SOS Emergency Broadcast	Android (App), Browser (Admin Web)	100%	0.8 seconds

Table 5.3: System Testing & Performance Summary

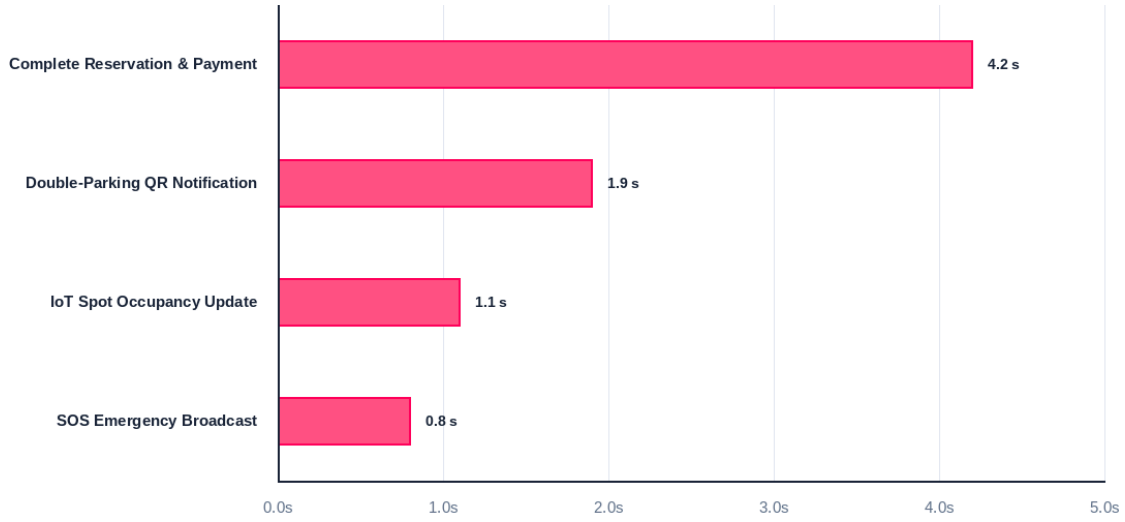


Figure 5.3: Response Time Graph in Seconds

5.3.4 User Acceptance Testing (UAT)

To evaluate the system's usability and user satisfaction, UAT was conducted with 15 participants, including university students, staff, and daily commuters. Since the IoT sensor system was only built as a tabletop prototype, participants focused on testing the mobile application. The application was provided as an APK and tested on their own Android smartphones.

UAT Methodology

Participants installed the myMove application on their Android devices using the APK file and were asked to complete several main tasks to test the app's features and user interface:

1. Register a new account and set up Two-Factor Authentication.
2. Add a vehicle's model, and plate number, then view the generated QR code.
3. Browse parking locations, check the parking fees, and book a simulated parking spot.
4. Scan a printed vehicle QR code using the phone's camera, open the anonymous scanner web page, and send a double-parking notification.
5. Activate an SOS alert and check that the system responds correctly.

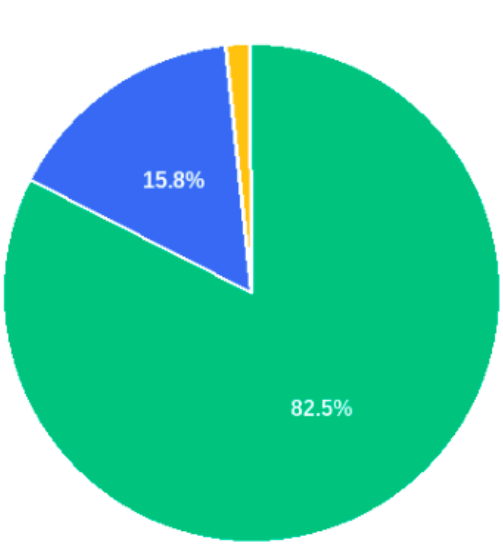
NOTE: The live demonstration of the ESP32 3d prototype was also shown to participants to demonstrate how real-time sensor data changes the parking spot status in the app.

Survey Results

The survey questions and scores from the 15 participants are shown in Table 5.4:

ID	Survey Statement	Score (Out of 5.0)
Q1	The mobile application interface is visually appealing, modern, and easy to navigate.	4.80
Q2	Registering a vehicle and generating the QR code is straightforward.	4.53
Q3	Scanning a QR code to notify a blocking vehicle owner is fast and highly intuitive.	4.47
Q4	The real-time interactive grid accurately reflects the parking slot availability.	5.00
Q5	Booking a spot and authorizing payment via Stripe card feels secure and convenient.	5.00
Q6	Configuring 2FA and editing system permissions is simple to execute.	4.80
Q7	The system protects user privacy by eliminating the need to display phone numbers on the vehicle	5.00
Q8	Overall, myMove is a highly effective solution compared to traditional parking systems.	4.87

Table 5.4: User Acceptance Testing (UAT) Survey Results



RESPONSE BREAKDOWN

Rating Option	Count	Percentage
5 - Strongly Agree	99	82.5%
4 - Agree	19	15.8%
3 - Neutral	2	1.7%
2 - Disagree	0	0.0%
1 - Strongly Disagree	0	0.0%
Total	**120**	**100.0%**

Figure 5.4: Overall User Acceptance Testing (UAT) Results from Q1 - Q8

5.4 Feedback & Discussion

The feedback collected from the survey and interviews after testing was mostly positive. The participants response highlighted several main strengths of the system, as well as some areas that could be improved.

Key Appreciated Features:

1. **Private QR Communication:** Participants liked the QR feature because it allows drivers to contact each other without sharing personal phone numbers. This helps protect their privacy during double-parking situations.
2. **Easy Access Guests:** Users liked that guests can scan the QR code and send notifications without installing the mobile app. This makes the system easier and more convenient to use.
3. **Real-Time Parking Updates:** Participants liked how the ESP32 prototype updates parking availability in the mobile app in real time, making it easier to find available parking spaces.
4. **Easy Vehicle Management & Payment:** Participants liked being able to add multiple vehicles to one profile. They also found the Stripe payment process smooth and quick.

Areas for Improvement:

1. **Functional & Security Improvement**: Add custom emergency contacts, such as family members or friends, to the SOS feature. A countdown timer is also shown on the homepage when a parking spot is reserved.
2. **Navigation & Booking Convenience:** Google Maps or Waze to guide users to their reserved parking spots and "Save Favorite Slots" for users who regularly use the same parking spots.
3. **Payment Options:** Adding e-wallets such as Touch 'n Go eWallet and QR-code payments.
4. **User Interface & Visual Options**: Adding a Light Mode toggle to let users choose their preferred display mode.
5. **Physical & Environmental Considerations:** Replacing ultrasonic sensors with camera-based sensors for better outdoor use and detection of number plate (ALPR). The app could also allow users to report physical obstacles or damaged sensors in a parking spot.
6. **Accessibility:** Adding a Bahasa Melayu language option to make the app easier for local drivers.

5.5 Discussion of Practical Implication

The results from the system testing and UAT provide useful information for different groups involved:

1. The high score for QR communication (Q7, Mean: 5.00) shows that myMove can reduce double parking by providing a simple and secure way to contact vehicle owners.
2. The fast sensor updates (1.1s) and SOS alerts (0.8s) show that it can provide real-time information and help users respond quickly to emergencies.
3. The simple scanner web page allows guest users to access features without needing to install the full mobile application.
4. The project provides an example of connecting ESP32, Firebase, and Stripe to develop a smart parking system.

5.6 System Limitation

Even though the evaluation shows that the myMove system is stable and works properly, the following technical limitations is identified:

1. **Internet Dependency:** Real-time updates and QR communication need an active internet connection. Poor internet connectivity may cause delays.
2. **Language Limitation:** The app currently supports only English, which may make it less convenient for drivers who doesn't understand the language
3. **Prototype Limitations:** The ultrasonic sensors were tested only in a 3d prototype and may not perform as well in outdoor conditions.

5.7 Summary

Chapter 5 shows the complete testing and evaluation results of the myMove. Unit testing checked the individual parts and achieved a 97.8% pass rate, while integration testing confirmed that the hardware, backend database, and mobile app worked together successfully with a 100% success rate. System testing showed fast response times, with parking sensor updates appearing in 1.1 seconds and double-parking notifications arriving in less than 2 seconds. Finally, User Acceptance Testing with 15 participants showed that the system was easy to use and that users were satisfied with its privacy features. Overall, the results show that the system works properly, provides good security, and has the potential to solve real-world parking problems.

CHAPTER 6

CONCLUSION

6.1 Introduction

This chapter wraps up the thesis by summarizing the myMove project. It reviews the main reasons for developing the system and the achievements, explains how the research objective was completed, discusses the technical and practical contributions of the project, describes current limitations, and provides suggestions for future improvements. Parking congestion, difficulty finding available spaces, privacy concerns when leaving contact details on vehicles, and double-parking problems continue to affect daily commuters in Malaysia. The myMove system was developed as a combination of IoT hardware and software platforms to address these problems by providing real-time parking updates, parking reservations, secure card payments, emergency SOS alerts, and private QR-based communication between drivers.

6.2 Project Summary

The myMove project followed the traditional Waterfall software development method, moving step by step from the initial literature review and user requirements to system design, hardware setup, software development, cloud deployment, and final system testing.

The complete system consists of five main parts that work together:

1. **myMove Flutter Mobile Application:** Tested on an Android smartphone, providing drivers with real-time parking spot views, parking reservations, Stripe credit card payments, vehicle profile management, Two-Factor Authentication (2FA), and an in-app driver communication feature.
2. **myMove Web Admin Dashboard:** Created using React, TypeScript, and Vite for parking lot management and security. It supports add parking location and slot management, live parking monitoring, revenue tracking, and instant SOS emergency alerts.
3. **myMove Scanner-Web Guest Portal:** A mobile web interface that can be used by scanning a vehicle's windshield QR code. It allows guest drivers to notify vehicle owners about double-parking situations without installing the mobile app.

4. **IoT Hardware Tabletop Prototype:** A 3D prototype provided with an ESP32, HC-SR04 ultrasonic sensors, and custom C++ firmware. It detects changes in parking occupancy and sends real-time updates to the cloud.
5. **Serverless Cloud Infrastructure:** Powered by Firebase, including Authentication with 2FA TOTP, Cloud Firestore for user, vehicle, and booking records, Realtime Database for fast sensor updates, and Cloud Functions for Stripe payments and FCM push notifications.

Through several levels of testing, including unit testing (97.8% pass rate), integration testing (100% pass rate), system performance testing (response times below 2.0s), and User Acceptance Testing (UAT) with 15 real users (overall score of 4.81 out of 5.00), the myMove platform has shown good performance, reliability, and ease of use as a smart parking solution.

6.3 Achievement of Project Objectives

The main goal of this research was to develop and evaluate a smart parking and communication system that can reduce driver stress, parking search time, and problems caused by double parking. The project successfully achieved all three research objectives defined in Chapter 1:

1. Research Objective 1:

"To study existing parking systems, identify current issues faced by Malaysian drivers, analyze the limitations of current car parking application, define user requirement specification and features through literature review and survey"

- **Status:** Fully Achieved
- **Outcome:** A review of existing parking solutions, such as JomParking and Flexi Parking, was mentioned to identify their limitations. These included the lack of real-time parking spots, privacy concerns caused by leaving contact numbers on vehicles, and delays in payment confirmation. User requirements were then collected to define the system's functional and non-functional requirements, as well as the main architecture, database structure, and UI designs presented in Chapter 4.

2. Research Objective 2:

“To develop the myMove application (mobile and web admin) by implementing its core features, including real-time parking search, hardware-integrated spot monitoring, and booking.”

- **Status:** Fully Achieved
- **Outcome:** The complete myMove system was developed and successfully deployed. The main features include:
    - **IoT Hardware Integration:** ESP32 / HC-SR04 sensor sends parking spot occupancy updates to Firebase RTDB in real time, with an average response time of 1.1 seconds.
    - **Mobile App & Admin Page:** Drivers can search for parking, reserve slots, and make payments, while admin can manage parking facilities and receive SOS alerts.
    - **Privacy-Preserving Contact:** The scanner web page allows guests to scan a vehicle QR code and send instant notifications to the vehicle owner without showing personal phone numbers.

3. Research Objective 3:

"To evaluate the application usability and user satisfaction by conducting User Acceptance Testing (UAT) with real users and collecting feedback for further improvement."

- **Status:** Fully Achieved
- **Outcome:** The system was tested through unit testing, integration testing, and system performance testing, followed by UAT with 15 participants, including students, staff, and daily commuters. The UAT survey achieved an average score of 4.81 out of 5.00. All participants gave positive ratings for real-time parking grid accuracy (Q4, Mean: 5.00), payment security (Q5, Mean: 5.00), and privacy protection (Q7, Mean: 5.00). The written feedback also showed high satisfaction and provided useful suggestions for future improvements.

A summary showing each research objective and its achievements is presented in Table 6.1 below:

Research Objective	Status	Key Deliverable	Result
RO1: Study existing systems, identify issues, and define user requirements.	Achieved	Literature review, requirement specification documents, and architecture design.	Identified functional gaps in existing apps; established full PRD specs.
RO2: Develop myMove mobile app, web admin dashboard, and IoT spot monitoring.	Achieved	Flutter app, React Web Admin, Scanner-Web, ESP32 prototype, Firebase backend.	100% feature completion across mobile, web, backend, and hardware integration.
RO3: Evaluate system usability, technical performance, and user satisfaction via UAT.	Achieved	Unit/Integration/System tests, 15-participant UAT survey, and qualitative analysis.	97.8% Unit Pass Rate, 100% Integration Pass Rate, 4.81 / 5.00 UAT Satisfaction.

Table 6.1: Summary of Research Objectives

6.4 Contribution of Systems

The development and evaluation of the myMove system provide several practical, technical, and academic contributions:

1. Contribution to Drivers & Commuters:
    - **Privacy Double-Parking Solution:** Removes the need to display personal phone numbers on vehicle windshields by using a secure QR code system for contacting vehicle owners.
    - **Reduced Parking Search Stress:** Real-time parking spot information helps drivers find and reserve available spaces more easily, which can also reduce unnecessary driving and traffic congestion.
    - **Enhanced In-App Security:** Provides Two-Factor Authentication (TOTP) and an SOS emergency button to improve user security and safety within parking areas.

2. Contribution to Parking Lot Operators & Facility Management:
    - **Real-Time Monitoring:** Gives admin live information about parking occupancy and system activity without needing to manually check each parking area.
    - **Faster Emergency Response:** SOS alerts trigger the alarm and show the user's exact location on the admin dashboard within 0.8 seconds, helping staff respond more quickly.
    - **Centralized Parking Management:** Allows operators to easily change parking rates, add or manage parking spaces, and view revenue information through one web dashboard.

3. Contribution to Software & UX Engineering:
    - **Zero-Install Guest Interaction:** Shows how a web page can be used by guest users without requiring them to download and install the mobile application.
    - **Multi-Platform Ecosystem Integration**: Provides an example of how mobile applications, web dashboards, and Firebase cloud services can be combined to build one connected system.

4. Academic & Technical Contribution:
- **Low-Cost IoT-Cloud Hardware**: Provides an example of using affordable ESP32 with Firebase RTDB to achieve parking status updates in under 1.5 seconds, making it suitable for smart city.

6.5 System Limitation

While myMove successfully meets the main functional and performance, the following limitations were identified during testing:

1. **Dependence on Internet Connectivity:** Real-time sensor updates, Stripe payments, and notifications requires internet connection. In underground or basement parking areas with weak network coverage, updates may be delayed.
2. **Prototype Sensor Limitations:** The 3d prototype uses HC-SR04 ultrasonic sensors, which are affordable and suitable for tabletop testing but may be affected by rain, dust, and temperature changes when used in outdoor parking areas.
3. **Language:** Mobile application, web dashboard, and notifications currently support only English. This may make it less accessible to drivers in Malaysia who are more comfortable using other languages.

6.6 Future Enhancement & Recommendation

Based on the user feedback collected from the development phase, the following improvements are recommended for future improvement:

- **Computer Vision & License Plate Recognition (ALPR):** Replace ultrasonic sensors with AI cameras that can detect parking occupancy and recognize license plates.
- **Magnetic and Radar Sensors:** Use ground-based magnetic or radar sensors in outdoor parking areas. These sensors can be more suitable for different weather conditions.
- **AR & Indoor Navigation:** Add AR camera guidance or indoor parking maps to help drivers find their reserved parking spots.
- **Local Payment Options:** Add Malaysian payment methods such as Touch 'n Go eWallet, FPX Online Banking, and DuitNow QR.
- **Prepaid myMove Wallet:** Allow users to add money in the app and use the balance for parking payments without entering their card details.
- **Multi-Language Support:** Add Bahasa Melayu and Mandarin language options to the app, admin dashboard, and scanner web page.
- **Countdown Timers & Automatic Slot Release:** Add countdown timers to show the remaining reservation time and automatically make a reserved slot available again if the driver does not arrive within a 15-minute grace period.
- **Emergency Contact Alerts:** Allow users to add emergency contacts, such as family members or friends, who can receive SMS alerts when the SOS button is triggered.

6.7 Summary

Chapter 6 shows the thesis by summarizing the overall project scope and deliverables. It also confirmed that all three research objectives achieved based on the testing results, discussed the practical and contributions of the system, described the technical limitations, and suggested possible improvements for future development. With the completion of this chapter, the myMove thesis report is complete and provides a clear summary of the project's development, results, and future potential.

REFERENCES

Zhen C., Z. Xu, Kang T., S. Jia (2025) *Environmental and Social Benefits of Urban Parking Space Shortages Mitigation Management Model: A System Dynamics and Nudge Approach*. 17(14), 6414–6414.

Group, A., & Group, A. (2024, December 23). *Efficient Parking Management Systems for Smart Cities - Abdul Rahman Al Shareef Group*. Abdul Rahman al Shareef Group.

Khalid, H. G., Abbas, H. H., Hussein, M. J., Mahmood Jawad Abu-AlShaeer, Mukhlif, S. K., & Dmytro Khlaponin. (2024). *Unveiling the Next Frontier: The Future of Connected Technologies*. 207–218.

Puspitasari, D., Noprianto, N., Afif Hendrawan, M., & Andrie Asmara, R. (2021). Development of smart parking system using internet of things concept. *Indonesian Journal of Electrical Engineering and Computer Science*, *24*(1), 611.

Kalašová, A., Čulík, K., Poliak, M., & Otahálová, Z. (2021). Smart Parking Applications and Its Efficiency. *Sustainability*, *13*(11), 6031.

Haslinda A., R. Ng, Rosmah M. (2016) *Factors Influencing Haphazard Parking in The Klang Valley, Malaysia 10.6007/IJARBSS/v6-i9/2303*

Anis.(2025) *KL Sees Over 1,000 Abandoned Cars in Just Five Months* https://www.carz.com.my/webview/2025/07/kl-sees-over-1000-abandoned-cars-in-just-five-months.

Aditya Kumar. (2022, December 20). *Has Technology Improved Our Lives? | Simplilearn*. Simplilearn.com; Aditya Kumar. https://www.simplilearn.com/how-has-technology-improved-our-lives-article

Najihah Mad Rosni, N., Fadzil Jobli, A., & Neging, M. (2019). Driver Parking Behaviour: An Observational and Experimental Intervention Study. *IOP Conference Series: Materials Science and Engineering*, *601*(1), 012002.

Sampath, L. P., & Boggavarapu, K. S. S. (2025). *IoT-based Smart Parking System for Industry 4.0 with QR code access and real-time navigation. EPJ Web of Conferences*, *336*, 03003.

Siti Aslinda Mahat, & Maslinda Mohd Nadzir. (2025). *ASSESSING USER SATISFACTION OF SMART PARKING PAYMENT SYSTEM IN SELANGOR*. *Journal of Digital System Development*, *3*(1), 13–28.

Lim, B. (2017, July 31). *Uni students develop smart system to curb double parking problems*. NST Online; New Straits Times. https://www.nst.com.my/lifestyle/bots/2017/07/262578/uni-students-develop-smart-system-curb-double-parking-problems